

AWS und Terraform

Deutsches Nachschlagewerk

*Praxisnahe Zusammenfassung der kompletten 10 Kurswochen
Tag 1 bis Tag 50*

DCI Cloud Engineering - persönliche Lernunterlage

Inhaltsübersicht

Vorwort

1. Das AWS-Grundmodell verstehen
2. Regionen, Availability Zones und Edge Locations
3. IAM: Identitäten, Rollen und Berechtigungen
4. IAM Roles, STS und Cross-Account Access
5. AWS Organizations, OUs und SCPs
6. EC2: Virtuelle Server in AWS
7. EC2-Netzwerk, Metadaten und Platzierung
8. EC2-Speicher: EBS und Instance Store
9. EC2-Kosten und Tenancy
10. VPC, CIDR und Subnetze
11. Internet, NAT und sichere Admin-Wege
12. Security Groups, NACLs und Network Firewall
13. Private AWS-Zugriffe, Peering und Hybrid Networking
14. VPC Flow Logs und CloudTrail
15. Amazon S3: Objektspeicher

16. Route 53 und DNS

17. Terraform und Infrastructure as Code

18. Terraform-Workflow

19. Terraform State, Remote Backend und Drift

20. Variablen, Outputs, Locals und sensible Werte

21. HCL-Ausdrücke, Funktionen und Iteration

22. Abhängigkeiten und Lifecycle

23. Terraform-Module

24. Import, Refactoring und Debugging

25. Workspaces und HCP Terraform

26. Gesamtarchitektur: Wie die Bausteine zusammenspielen

27. DevOps-Fokus: Welche Fähigkeiten zusätzlich wichtig sind

28. Prüfungs- und Praxismerksätze

29. Kompakte Befehlsübersicht

30. Glossar

Schluss

Vorwort

So ist die überarbeitete Version aufgebaut

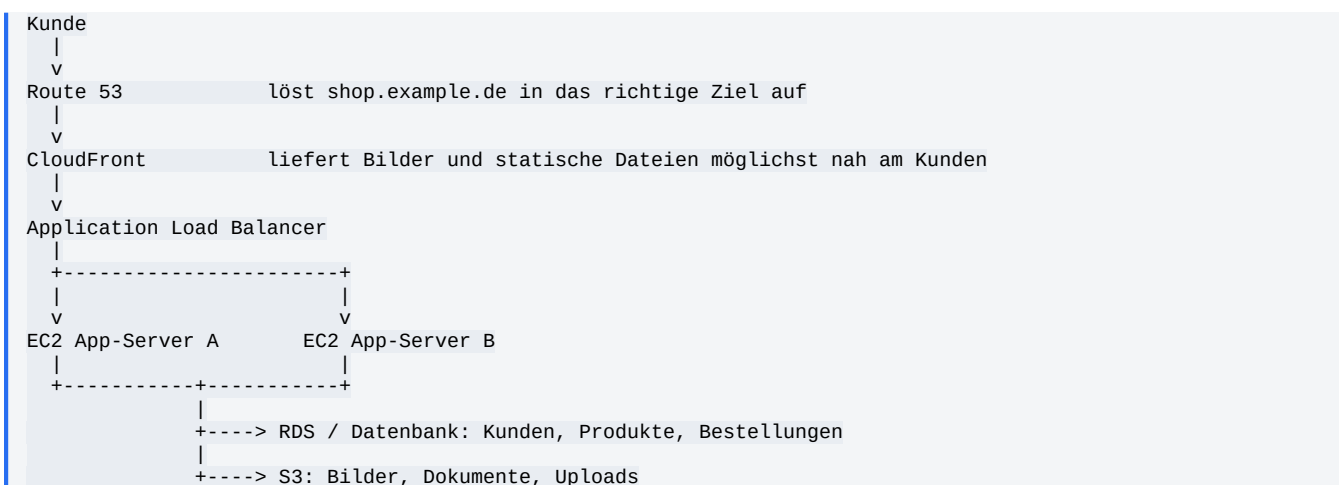
Die erste Fassung war bewusst kompakt. Diese Version geht einen Schritt weiter: Sie erklärt nicht nur, **was** ein AWS- oder Terraform-Begriff bedeutet, sondern auch, **warum** er gebraucht wird und **wie** er mit den anderen Bausteinen zusammenspielt.

Fast jedes Hauptthema folgt deshalb demselben Muster:

1. **Kurz erklärt:** Was ist der Dienst oder Begriff?
2. **Warum braucht man ihn?** Welches konkrete Problem löst er?
3. **Praxisbeispiel:** Was passiert in einer echten Anwendung?
4. **Zusammenspiel:** Welche anderen Dienste werden dafür benötigt?
5. **Fehlerbild:** Was passiert, wenn ein Baustein fehlt oder falsch eingestellt ist?
6. **Praxis oder CLI:** Ein kleiner Befehl oder Terraform-Ausschnitt, wenn er das Verständnis verbessert.
7. **Merksatz:** Die Kernaussage für Prüfung und Praxis.

Leitbeispiel: Ein kleiner Online-Shop

Durch das Buch zieht sich ein gemeinsames Beispiel. Der Shop besteht aus mehreren Schichten:



Dazu kommen IAM für Berechtigungen, VPC und Subnetze für das Netzwerk, CloudWatch und Flow Logs für die Fehlersuche sowie Terraform für die wiederholbare Bereitstellung.

Der wichtigste Gedanke dabei ist: **Keiner dieser Dienste ist die komplette Anwendung.** Jeder Dienst löst ein bestimmtes Problem. Eine gute Architektur entsteht erst dadurch, dass die Bausteine sauber zusammengesetzt werden.

Dieses Nachschlagewerk fasst die bereitgestellten Kursunterlagen **Tag 1 bis Tag 50** zusammen. Die Inhalte sind nicht einfach nach Unterrichtstagen kopiert, sondern fachlich neu sortiert. Dadurch werden Zusammenhänge sichtbar, die im Unterricht oft über mehrere Tage verteilt behandelt werden.

Die Quizwochen 9 und 10 wiederholen viele frühere Themen. Ihre zusätzlichen Terraform-Inhalte - unter anderem Lifecycle-Regeln, Funktionen, Workspaces, Troubleshooting und kontrollierte CI/CD-Abläufe - sind an den passenden Stellen eingearbeitet.

Ziel: Du sollst nachschlagen können, was ein Dienst technisch ist, wofür er verwendet wird, welche Verantwortung bei dir bleibt und woran du typische Prüfungsfragen erkennst.

Kursübersicht nach Wochen

Woche	Tage	Schwerpunkt
1	1-5	Cloud-Modelle, globale Infrastruktur, IAM, Rollen, STS, Organizations und SCPs
2	6-10	EC2, Netzwerkzugriff, EBS, Kostenmodelle, AMIs, IMDS und Placement Groups
3	11-15	VPC, CIDR, Subnetze, Routing, IGW, NAT, Bastion, Security Groups und NACLs
4	16-20	Sichere Managementpfade, VPC Endpoints, Peering, Transit Gateway, Hybrid und Flow Logs

Woche	Tage	Schwerpunkt
5	21-25	S3, Verschlüsselung, Versionierung, Lifecycle, Replikation und Route 53
6	26-30	IaC und Terraform-Grundworkflow: init, validate, plan und apply
7	31-35	Variablen, Outputs, HCL, Data Sources, Abhängigkeiten und Module
8	36-40	Eigene Module, Remote State, Debugging, Import, Drift und HCP Terraform
9	41-45	Prüfungswiederholung AWS-Grundlagen/IAM/EC2 plus Terraform-Vertiefung
10	46-50	Prüfungswiederholung Netzwerk/Speicher/Compute plus Funktionen, Lifecycle und Workspaces

1. Das AWS-Grundmodell verstehen

Praxis: Derselbe Shop in drei Betriebsmodellen

Angenommen, der Online-Shop braucht einen Webserver und eine Datenbank.

IaaS: Du mietest die Grundbausteine

Du startest eine EC2-Instanz, installierst Linux, Nginx, eine Laufzeitumgebung und deine Anwendung. Die Datenbank könnte ebenfalls auf einer eigenen EC2-Instanz laufen.

```
Du verwaltest:
Linux -> Updates -> Nginx -> Anwendung -> Datenbanksoftware -> Backups

AWS verwaltet:
Rechenzentrum -> Hardware -> Hypervisor -> physisches Netzwerk
```

Das passt, wenn du volle Kontrolle brauchst. Der Nachteil: Du musst auch fast alles selbst betreiben.

Managed Platform: AWS übernimmt Teile des Betriebs

Die Anwendung läuft weiterhin auf EC2 oder einer Plattform, die Datenbank aber auf RDS. AWS kümmert sich dann um die zugrunde liegende Datenbankinfrastruktur, während du Daten, Benutzer, Schema und Zugriffe verwaltest.

```
Du: Tabellen, Daten, Benutzer, Abfragen
AWS: Datenbankserver, Betriebssystem, Ersatzhardware, viele Wartungsabläufe
```

SaaS: Du benutzt eine fertige Anwendung

Für die Firmen-E-Mail nutzt du zum Beispiel einen fertigen Mailedienst. Du entwickelst und betreibst dafür keinen eigenen Mailserver.

Warum diese Unterscheidung wichtig ist

Bei jeder Architekturentscheidung tauschst du **Kontrolle gegen Betriebsaufwand**. Mehr Kontrolle ist nicht automatisch besser. Ein selbst betriebener Datenbankserver kann sinnvoll sein, wenn Spezialfunktionen oder tiefe Systemzugriffe nötig sind. Für einen normalen Shop ist ein Managed Service häufig sicherer und einfacher zu betreiben.

Praxisfrage: Brauche ich wirklich Zugriff auf das Betriebssystem, oder brauche ich nur die Funktion des Dienstes? Diese Frage führt oft zur richtigen Auswahl.

1.1 Cloud statt eigenes Rechenzentrum

Bei klassischer On-Premises-Infrastruktur kaufst, betreibst und wartest du Hardware selbst. In AWS mietest du Ressourcen über APIs. Das ändert nicht die technischen Grundlagen eines Servers, sondern vor allem **Bereitstellung, Skalierung und Verantwortung**.

Wichtige Cloud-Eigenschaften:

- Ressourcen können in Minuten erstellt und wieder entfernt werden.
- Abgerechnet wird überwiegend nach tatsächlicher Nutzung.
- Kapazität lässt sich vertikal und horizontal anpassen.
- Infrastruktur kann über CLI, SDK und Infrastructure as Code automatisiert werden.
- Ausfallsicherheit entsteht nicht automatisch. Sie muss durch Architektur geplant werden.

1.2 IaaS, PaaS und SaaS

Cloud-Modelle: Wer verwaltet welche Schicht?

Je weiter rechts, desto weniger Betriebsaufwand – aber auch weniger Kontrolle.



Prüfungslogik: EC2 = viel Kontrolle. Managed Services = weniger Betrieb. Fertige Anwendung = nur benutzen.

Cloud-Modelle und Verantwortlichkeiten

- **IaaS:** Virtuelle Infrastruktur als Baustein. Beispiel: EC2. Du erhältst einen virtuellen Server und verwaltest Betriebssystem, Patches, Software und Anwendung.
- **PaaS / Managed Platform:** AWS übernimmt mehr Betriebsaufgaben. Beispiele sind Elastic Beanstalk oder verwaltete Datenbankdienste. Du konzentrierst dich stärker auf Anwendung und Daten.
- **SaaS:** Eine fertige Anwendung wird direkt genutzt. Infrastruktur und Plattform sind für den Benutzer unsichtbar.

Wichtige Einordnung: AWS-Dienste passen nicht immer sauber in nur eine Schublade. Für Prüfungen zählt meistens die Frage: Wie viel musst du selbst verwalten? EC2 bedeutet viel Kontrolle, Managed Services weniger Betrieb und SaaS nur Nutzung.

1.3 Shared Responsibility Model

Konkretes Beispiel: Sicherheitsupdate auf dem Webserver

Bei einer EC2-Instanz stellt AWS sicher, dass die physische Hardware und die Virtualisierungsschicht funktionieren. Ein Sicherheitsupdate für das Linux-System innerhalb deiner EC2-Instanz ist aber deine Aufgabe.

Sicherheitslücke im AWS-Hypervisor	-> AWS muss reagieren
Sicherheitslücke in deinem Linux	-> du musst patchen
Öffentlicher Port 22 in der SG	-> deine Konfiguration
Defekte physische Festplatte	-> AWS-Infrastruktur

Bei S3 gibt es kein Betriebssystem, das du patchen musst. Trotzdem musst du verhindern, dass der Bucket öffentlich lesbar ist. Managed bedeutet also nicht verantwortungslos, sondern nur: **Die technische Grenze deiner Verantwortung verschiebt sich.**

AWS verantwortet die **Security of the Cloud:** Rechenzentren, physische Hardware, grundlegende Netzwerk- und Virtualisierungsschicht. Du verantwortest die **Security in the Cloud:** Identitäten, Berechtigungen, Daten, Konfigurationen, Betriebssysteme und Anwendungen - abhängig vom gewählten Dienst.

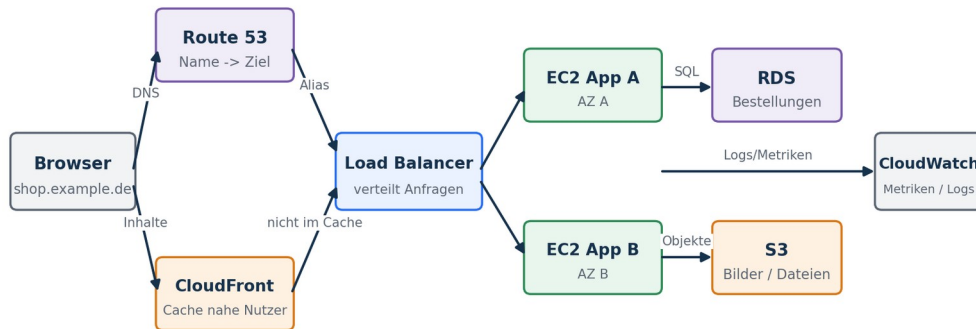
Beispiel	AWS	Du
EC2	Hardware und Hypervisor	Betriebssystem, Patches, Anwendung, Security Group
S3	Betrieb und Haltbarkeit des Speicherdienstes	Bucket-Policies, Verschlüsselungsauswahl, Datenklassifikation
Managed Datenbank	Plattform, Backups je Konfiguration, Infrastruktur	Benutzer, Daten, Schema, Zugriff und passende Einstellungen

Merksatz: Je verwalteter ein Dienst ist, desto mehr Betriebsarbeit übernimmt AWS. Die Verantwortung für Zugriffsrechte und Daten verschwindet aber nie.

2. Regionen, Availability Zones und Edge Locations

Praxisfluss: Ein Kunde öffnet den Online-Shop

Jeder Baustein löst ein anderes Problem. Zusammen entsteht ein vollständiger Request-Pfad.



Wichtig: Route 53 speichert keine Webseite. Der Load Balancer führt keine Anwendung aus. S3 ist keine relationale Datenbank. Jeder Dienst hat eine klare Aufgabe.

Praxisvisualisierung

Praxis: Warum der Shop zwei AZs und trotzdem nur eine Region nutzt

Der Shop richtet seine Hauptumgebung in Frankfurt ein. In einer AZ läuft ein App-Server, in einer zweiten AZ ein weiterer. Der Load Balancer kennt beide Server.

```

Region Frankfurt
|
+-- AZ A: Public Subnet + Private App-Subnet + EC2 A
|
+-- AZ B: Public Subnet + Private App-Subnet + EC2 B
  
```

Fällt nur AZ A aus, kann der Load Balancer Anfragen weiterhin an EC2 B senden. Läuft dagegen alles nur auf einem Server in einer AZ, ist diese AZ ein Single Point of Failure.

CloudFront löst ein anderes Problem. Ein Kunde in Spanien muss ein Produktbild nicht jedes Mal aus Frankfurt laden. Das Bild kann an einem Edge-Standort in Kundennähe zwischengespeichert werden.

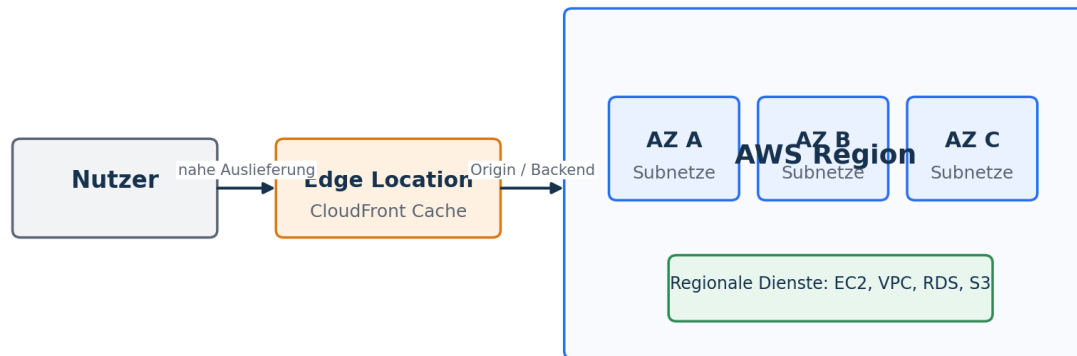
Was die drei Ebenen nicht leisten

- Eine zweite AZ schützt nicht vor einem vollständigen Regionsausfall.
- CloudFront ersetzt keinen App-Server und keine Datenbank.
- Eine zweite Region synchronisiert sich nicht automatisch. Daten, DNS, Deployments und Failover müssen geplant werden.

Sinn dahinter: AZs verteilen Ausfallrisiken innerhalb einer Region. Edge Locations verkürzen den Weg zum Nutzer. Multi-Region schützt gegen größere regionale Ausfälle oder erfüllt globale Anforderungen.

AWS Global Infrastructure

Regionen enthalten mehrere isolierte Availability Zones. Edge Locations liegen näher am Nutzer.



Multi-AZ = Hochverfügbarkeit innerhalb einer Region. Multi-Region = Schutz vor regionalen Großausfällen / Datenstandort-Anforderungen.

AWS Global Infrastructure

2.1 Region

Eine Region ist ein geografisch getrennter AWS-Standort. Viele Ressourcen sind regional. Gründe für die Auswahl einer Region sind:

- Nähe zu Benutzern und damit geringere Latenz
- gesetzliche Vorgaben und Datenstandort
- Verfügbarkeit bestimmter Dienste
- Kosten
- Disaster-Recovery-Strategie

Daten werden nicht automatisch in eine andere Region repliziert. Multi-Region ist eine bewusste Architekturentscheidung.

2.2 Availability Zone

Eine Availability Zone besteht aus einem oder mehreren voneinander isolierten Rechenzentren innerhalb einer Region. Die AZs einer Region sind über schnelle private Verbindungen gekoppelt.

- Ein Subnetz gehört immer genau zu **einer** AZ.
- Kritische Systeme sollten über mindestens zwei AZs verteilt werden.
- Multi-AZ schützt vor dem Ausfall einer einzelnen AZ, aber nicht vor einem vollständigen Regionsausfall.

2.3 Edge Location

Edge Locations werden unter anderem von CloudFront verwendet. Inhalte werden näher am Nutzer zwischengespeichert. Edge Locations sind keine normalen Regionen und keine Standorte, an denen du beliebige EC2- oder Datenbankressourcen startest.

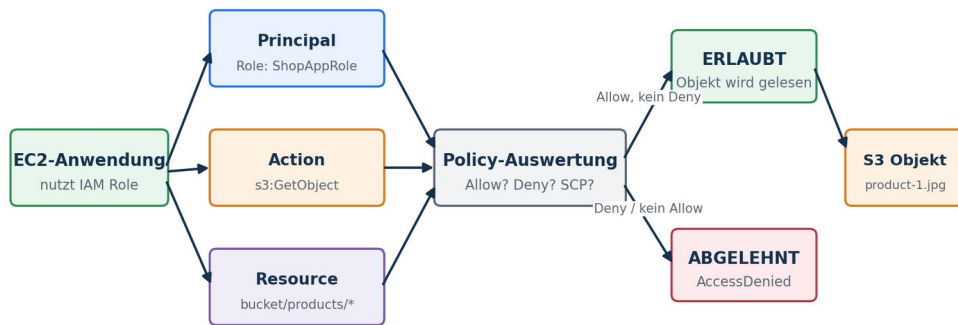
Prüfungsmerksatz:

- **AZ** = Hochverfügbarkeit innerhalb einer Region
- **Edge Location** = geringe Latenz bei der Inhaltsauslieferung
- **Multi-Region** = regionale Ausfallsicherheit, globale Nutzer oder Datenstandort

3. IAM: Identitäten, Rollen und Berechtigungen

Praxisbeispiel IAM: Darf die Anwendung ein Objekt aus S3 lesen?

AWS prüft Identität, Aktion, Ressource und alle Begrenzungen.



Konkreter Sinn: Die Anwendung bekommt nicht „S3-Zugriff allgemein“, sondern nur die API-Aktion auf einen festgelegten Pfad. Das begrenzt den möglichen Schaden.

Praxisvisualisierung

Praxis: Ein Entwickler, eine Anwendung und ein S3-Bucket

Im Shop gibt es drei verschiedene Akteure:

- **Julian als Entwickler** meldet sich über eine menschliche Identität an.
- **Die EC2-Anwendung** nutzt eine IAM Role.
- **Der S3-Bucket** besitzt eine Ressource, auf die nur bestimmte Aktionen erlaubt werden.

Die Anwendung soll Produktbilder lesen, aber keine Buckets löschen und keine IAM-Benutzer anlegen.

```

Principal: ShopAppRole
Action:    s3:GetObject
Resource:  arn:aws:s3:::shop-products/*
Resultat:  Bilder lesen erlaubt, alles andere standardmäßig verboten
  
```

Das ist der praktische Sinn von Least Privilege: Die Anwendung bekommt nicht „AWS-Zugriff“, sondern eine kleine, exakt definierte Fähigkeit.

Begriffe, die in Policies immer wieder vorkommen

- **Principal:** Wer stellt die Anfrage?
- **Action:** Welche AWS-API-Aktion soll ausgeführt werden?
- **Resource:** Auf welches konkrete Objekt oder welchen Dienst bezieht sich die Aktion?
- **ARN:** Eindeutiger AWS-Name einer Ressource.
- **Condition:** Unter welcher zusätzlichen Bedingung gilt die Regel?

Eine Security Group kann dieses Problem nicht lösen. Sie regelt Netzwerkverkehr. IAM regelt, ob eine authentifizierte Identität eine AWS-API-Aktion ausführen darf.

3.1 Authentifizierung und Autorisierung

- **Authentifizierung:** Wer bist du?
- **Autorisierung:** Was darfst du tun?

IAM ist ein globaler Dienst. Fast jede AWS-API-Anfrage wird gegen Berechtigungen geprüft.

3.2 Root Account, User, Group und Role

Element	Zweck	Zugangsdaten
Root Account	Eigentümeridentität des AWS-Kontos; nur für Sonderfälle	dauerhaft, besonders kritisch
IAM User	dauerhafte Identität, oft für einzelne Personen oder ältere Workloads	Passwort und/oder Access Keys
IAM Group	fasst Benutzer für gemeinsame Policies zusammen	keine eigene Anmeldung
IAM Role	temporär übernommener Berechtigungssatz	temporäre STS-Credentials

Der Root Account gehört nicht in den Alltag, MFA ist Pflicht. Für Unternehmenszugriffe sind zentraler Login, Federation oder Identity Center besser als gemeinsam genutzte IAM-User.

3.3 Policies

Eine Policy ist ein JSON-Dokument. Die wichtigsten Felder sind:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::example-bucket/*",
      "Condition": {
        "Bool": {"aws:SecureTransport": "true"}
      }
    }
  ]
}
```

- **Effect:** Allow oder Deny
- **Action:** API-Aktion
- **Resource:** Zielressource, meist als ARN
- **Condition:** zusätzliche Bedingung, zum Beispiel MFA, Quell-IP oder Tags
- **Principal:** steht vor allem in ressourcenbasierten Policies und Trust Policies

Managed Policy und Inline Policy

- **AWS Managed Policy:** von AWS gepflegt, schnell einsetzbar, oft breiter als nötig.
- **Customer Managed Policy:** von dir erstellt, wiederverwendbar und versionierbar.
- **Inline Policy:** fest in genau einem User, einer Group oder einer Role eingebettet; wird mit dieser Identität gelöscht.

Für saubere Umgebungen sind wiederverwendbare Customer Managed Policies meistens übersichtlicher als viele verteilte Inline Policies.

3.4 Auswertungslogik

Schritt für Schritt: Warum `AccessDenied` entsteht

Die EC2-Anwendung ruft `s3:GetObject` auf.

8. AWS erkennt die Rolle der EC2-Instanz.
9. AWS sucht alle relevanten Identitäts-, Ressourcen- und Begrenzungspolicies.
10. Ein explizites Deny beendet die Prüfung mit Ablehnung.
11. Gibt es kein Deny, muss mindestens ein passendes Allow vorhanden sein.
12. Fehlt ein Allow, gilt der Standard: abgelehnt.

Ein typischer Fehler ist eine Policy, die zwar `s3:GetObject` erlaubt, aber nur auf den Bucket selbst zeigt:

```
"Resource": "arn:aws:s3:::shop-products"
```

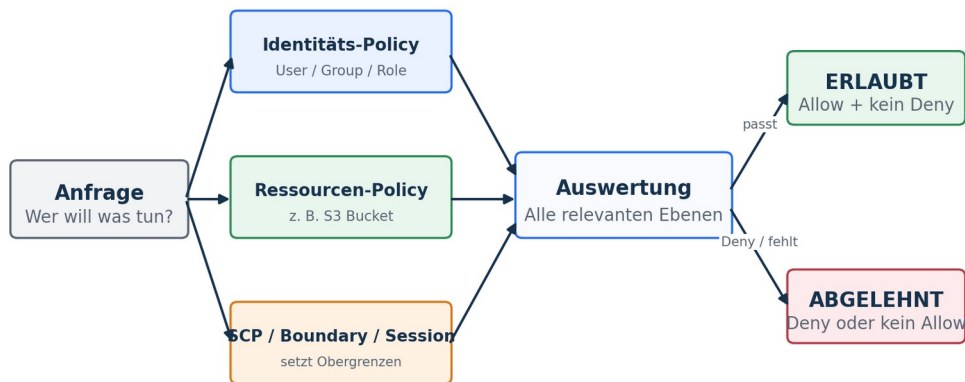
Für Objekte wird dagegen meist der Pfad mit `/*` benötigt:

```
"Resource": "arn:aws:s3:::shop-products/*"
```

Der Dienstname ist also richtig, die Ressource aber falsch adressiert. Genau solche Details erklären viele scheinbar unerwartete Berechtigungsfehler.

IAM-Auswertung: Warum wird eine Anfrage erlaubt oder abgelehnt?

Die stärkste Regel lautet: Ein explizites Deny gewinnt immer.



IAM-Auswertungslogik

Grundregeln:

13. Standardmäßig ist alles abgelehnt.
14. Ein explizites Allow kann eine Aktion erlauben.
15. Ein explizites Deny überschreibt jedes Allow.
16. SCPs, Permission Boundaries und Session Policies können die maximalen Rechte weiter begrenzen.

3.5 Least Privilege und MFA

Least Privilege bedeutet: nur die Aktionen, Ressourcen und Bedingungen freigeben, die tatsächlich benötigt werden. Ein Action: "*" und Resource: "*" ist bequem, vergrößert aber den möglichen Schaden bei Fehlern oder kompromittierten Zugangsdaten.

MFA ergänzt ein Passwort um einen zweiten Faktor. Sensible Aktionen können zusätzlich über eine Policy-Bedingung an eine MFA-authentifizierte Session gebunden werden.

4. IAM Roles, STS und Cross-Account Access

Praxis: GitHub Actions deployt in das Produktionskonto

Der Quellcode liegt in GitHub. Beim Merge soll eine Pipeline neue Infrastruktur oder eine Anwendung in AWS bereitstellen. Feste Access Keys im Repository wären gefährlich.

Besser ist dieser Ablauf:

```

GitHub Actions
|
| authentifiziert sich über einen vertrauenswürdigen Mechanismus
v
AWS STS
|
| gibt kurzlebige Session-Credentials aus
v
DeployRole im Produktionskonto
|
| darf nur definierte Deployment-Aktionen ausführen
v
EC2 / S3 / Terraform-Backend / andere Zielressourcen
  
```

Die Trust Policy legt fest, dass der definierte GitHub-Workflow die Rolle übernehmen darf. Die Permission Policy legt fest, welche Aktionen die Session anschließend ausführen darf.

Warum zwei Policies nötig sind

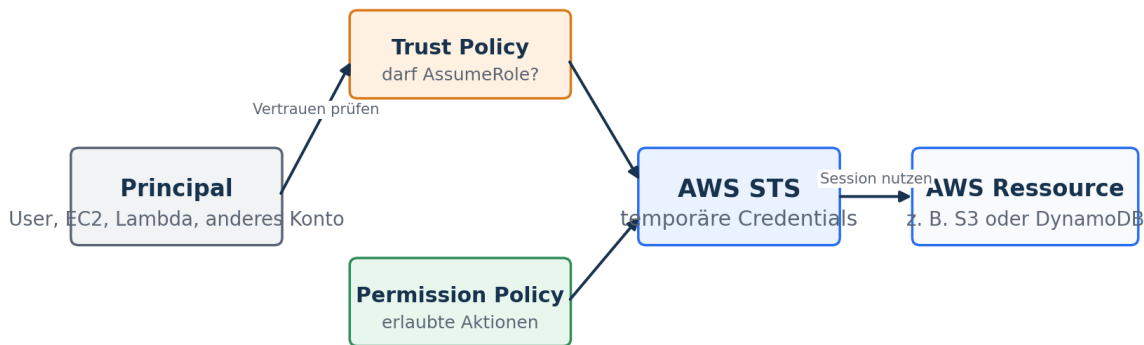
Die Trust Policy ist die Eingangstür zur Rolle. Die Permission Policy ist der Schlüsselbund, den du nach dem Eintritt bekommst.

- Trust stimmt, Permission fehlt: Rolle kann übernommen werden, API-Aufrufe scheitern.
- Permission wäre ausreichend, Trust stimmt nicht: Rolle kann gar nicht übernommen werden.

Temporäre Credentials begrenzen den Zeitraum eines möglichen Missbrauchs. Sie ersetzen aber keine sauber eingeschränkten Berechtigungen.

IAM Role und STS: Temporärer Zugriff ohne feste Schlüssel

Trust Policy beantwortet „Wer darf die Rolle übernehmen?“. Permission Policy beantwortet „Was darf die Rolle tun?“.



Cross-Account: Die Zielrolle muss dem Aufrufer vertrauen, und der Aufrufer braucht die Berechtigung sts:AssumeRole.

IAM Role und STS

4.1 Rolle statt fester Schlüssel

Eine Rolle besitzt keine permanenten Access Keys. Sie wird von einem vertrauenswürdigen Principal übernommen. AWS STS erstellt anschließend zeitlich begrenzte Credentials aus Access Key, Secret Key und Session Token.

Typische Rollen:

- EC2 Instance Role für Zugriff auf S3
- Lambda Execution Role für DynamoDB oder CloudWatch Logs
- Deployment Role für CI/CD
- Cross-Account Audit Role für ein zentrales Security-Konto

4.2 Trust Policy und Permission Policy

- **Trust Policy:** Wer darf die Rolle übernehmen?
- **Permission Policy:** Was darf die Rolle nach der Übernahme tun?

Wenn eine Rolle übernommen werden kann, die späteren API-Aufrufe aber abgelehnt werden, fehlt häufig die passende Permission Policy.

4.3 Cross-Account

Bei kontoübergreifendem Zugriff müssen beide Seiten zusammenpassen:

17. Die Zielrolle oder Zielressource vertraut dem fremden Principal.
18. Der aufrufende Principal darf die erforderliche Aktion ausführen, etwa sts:AssumeRole.
19. SCPs und explizite Deny-Regeln dürfen den Zugriff nicht blockieren.

Ein `ExternalId` schützt bei Zugriffen durch Drittanbieter vor dem sogenannten Confused-Deputy-Problem. Session Policies können die Rechte einer einzelnen Session weiter reduzieren, aber nie über die Rollenrechte hinaus erweitern.

5. AWS Organizations, OUs und SCPs

Praxis: Warum Entwicklung und Produktion getrennte Konten bekommen

Der Shop startet klein in einem AWS-Konto. Später arbeiten mehrere Personen daran. Ein Entwickler testet ein Terraform-Skript und löscht versehentlich eine Ressource. Befindet sich die Produktion im selben Konto und besitzt der Entwickler zu breite Rechte, kann der Schaden groß werden.

Eine typische Aufteilung sieht so aus:

```

AWS Organization
|
+-- Security OU
|   +-- Log-Archive Account
|   +-- Security-Tools Account
|
+-- Non-Production OU
|   +-- Development Account
  
```

```

|   +-- Test Account
|   |
|   +-- Production OU
|       +-- Shop Production Account

```

Die Kontentrennung schafft eine harte Grenze. Ein Fehler im Development Account verändert nicht automatisch Ressourcen im Production Account.

Ein SCP könnte im Production OU beispielsweise verhindern, dass CloudTrail deaktiviert oder eine nicht freigegebene Region verwendet wird. Das SCP gibt dem Entwickler aber keine Rechte. Es legt nur fest, was selbst mit einer IAM-Erlaubnis maximal möglich wäre.

Merksatz: IAM verteilt Schlüssel innerhalb eines Kontos. SCPs legen fest, welche Türen in einem ganzen Bereich überhaupt geöffnet werden dürfen.

5.1 Warum mehrere Konten?

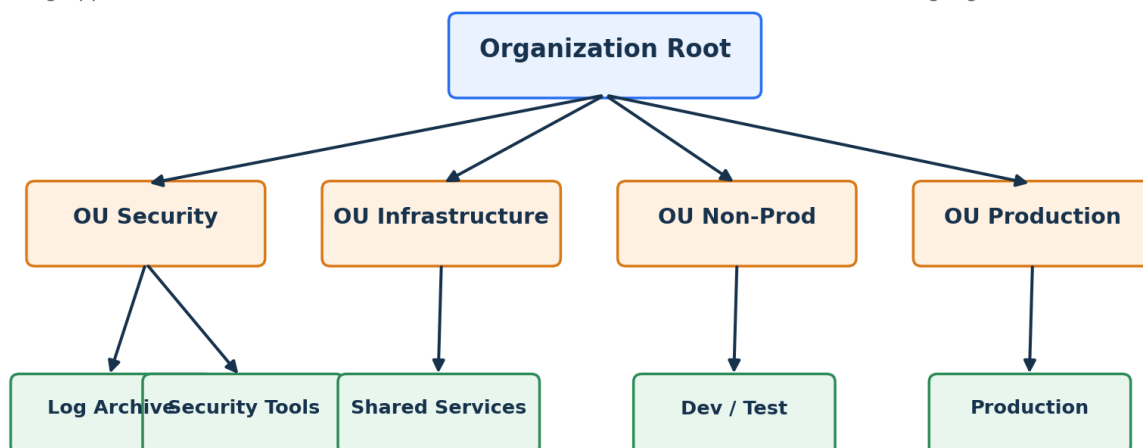
Mehrere AWS-Konten schaffen stärkere Grenzen als nur mehrere Benutzer in einem Konto:

- geringerer Blast Radius
- getrennte Kosten und Budgets
- getrennte Entwicklungs-, Test- und Produktionsumgebungen
- zentrale Sicherheits- und Logging-Konten
- klarere Compliance-Grenzen

5.2 Organisationsstruktur

AWS Organizations: Konten und Guardrails

OUs gruppieren Konten. SCPs werden nach unten vererbt und setzen die maximalen Berechtigungen.



SCP = Obergrenze. IAM im Mitgliedskonto erteilt die tatsächlichen Rechte innerhalb dieser Grenze.

AWS Organizations und OUs

- **Management Account:** verwaltet die Organisation und die konsolidierte Abrechnung.
- **OU:** logischer Container für Konten.
- **SCP:** setzt die maximal erlaubten Aktionen für Root, OU oder Mitgliedskonto.

5.3 SCP ist keine Berechtigung

Ein SCP erteilt keinen Zugriff. Es setzt nur die Obergrenze. Ein User kann eine Aktion nur ausführen, wenn seine IAM-Rechte sie erlauben und kein SCP sie blockiert.

Typische Guardrails:

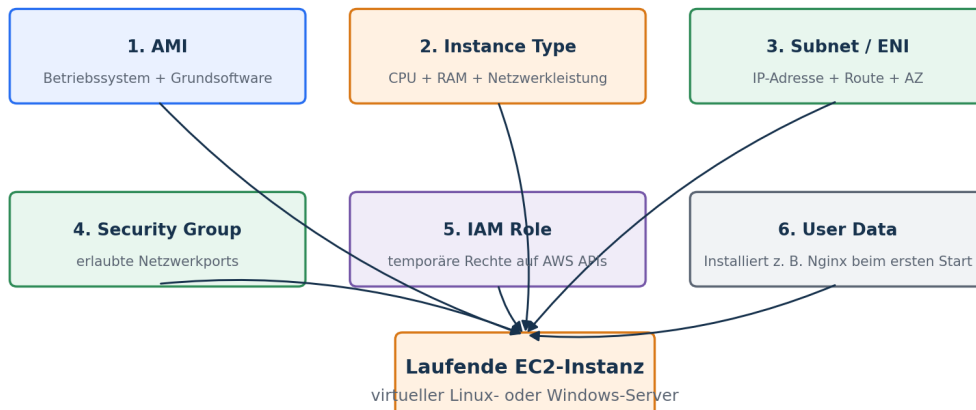
- Nutzung unerwünschter Regionen blockieren
- Deaktivieren von CloudTrail oder GuardDuty verhindern
- `organizations:LeaveOrganization` sperren
- riskante Dienste in Sandbox-Konten begrenzen
- Löschen zentraler Logs verhindern

Merksatz: SCP setzt die Decke, IAM öffnet die Tür.

6. EC2: Virtuelle Server in AWS

EC2-Start: Was AWS beim Launch zusammensetzt

Eine EC2-Instanz ist nicht nur „ein Server“. Mehrere Bausteine ergeben die konkrete Maschine.



Praktische Folge: Fehlt nur ein Baustein, kann der Server trotzdem „running“ sein und dennoch nicht erreichbar, nicht berechtigt oder falsch konfiguriert sein.

Praxisvisualisierung

Praxis: Was beim Start eines Shop-Servers wirklich zusammenkommt

Beim Klick auf „Launch instance“ wird nicht nur ein anonymer Server erzeugt. AWS setzt mehrere Entscheidungen zusammen:

20. **AMI:** Welches Betriebssystem und welcher Grundzustand werden gestartet?
21. **Instance Type:** Wie viel CPU, RAM und Netzwerkleistung stehen zur Verfügung?
22. **Subnetz und ENI:** In welcher AZ und mit welcher privaten IP befindet sich der Server?
23. **Security Group:** Welche Netzwerkverbindungen sind erlaubt?
24. **IAM Role:** Welche AWS-APIs darf die Anwendung aufrufen?
25. **EBS:** Welche dauerhaften Festplatten werden angehängt?
26. **User Data:** Welche Einrichtung läuft beim ersten Start automatisch?

Eine Instanz kann den Status `running` haben und trotzdem unbrauchbar sein. Beispiele:

- Kein Weg über die Route Table: Der Server hat kein funktionierendes Netzwerkziel.
- Port 443 fehlt in der Security Group: Der Webserver läuft, ist aber nicht erreichbar.
- User Data ist fehlerhaft: Die Instanz läuft, Nginx wurde aber nie installiert.
- IAM Role fehlt: Die Anwendung läuft, kann jedoch keine S3-Bilder lesen.

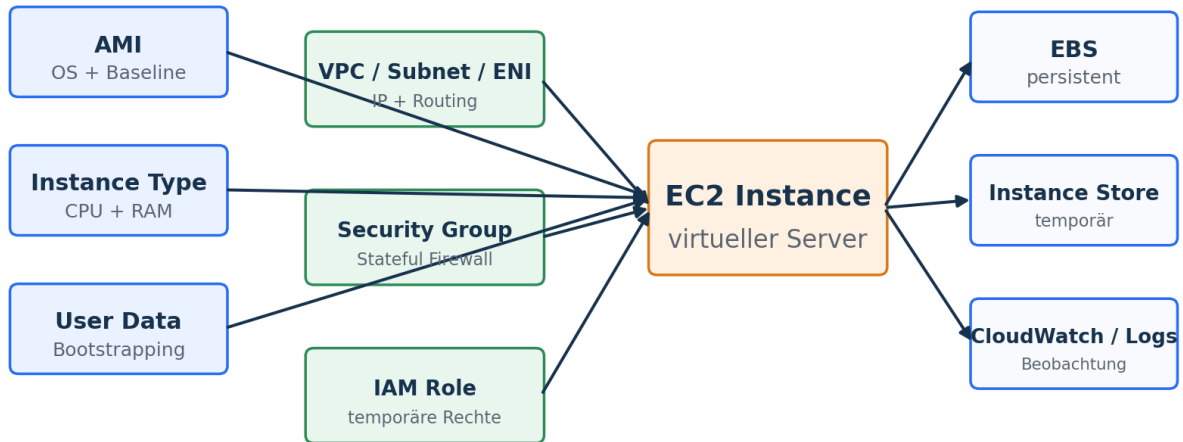
Kleines Bootstrap-Beispiel

```
#!/bin/bash
dnf update -y
dnf install -y nginx
systemctl enable --now nginx
echo "Shop-Server ist bereit" > /usr/share/nginx/html/index.html
```

User Data eignet sich für einen ersten automatisierten Aufbau. Bei größeren Systemen werden häufig Images, Konfigurationsmanagement oder Container verwendet, damit die Bereitstellung kontrollierter und schneller abläuft.

EC2 als virtueller Server: Bausteine eines Starts

EC2 ist Rechenleistung. AMI, Instanztyp, Netzwerk, Speicher und Rolle bestimmen die konkrete Maschine.



Bausteine einer EC2-Instanz

6.1 Was ist EC2?

EC2 stellt virtuelle Server bereit. Du bestimmst unter anderem:

- AMI und Betriebssystem
- CPU- und RAM-Größe
- Netzwerk und IP-Adressen
- Security Groups
- EBS-Volumes
- IAM Role
- Startskripte und Tags

EC2 ist IaaS. AWS betreibt die physische Infrastruktur, du verwaltest das Gastbetriebssystem und deine Software.

6.2 Instanzfamilien und Namen

Auswahl an einem konkreten Workload

Der Shop besteht aus unterschiedlichen Aufgaben:

Aufgabe	Typisches Profil	Sinnvolle Richtung
kleiner Webserver mit wechselnder Last	kurzzeitig mehr CPU, sonst wenig	burstable T-Familie
normaler App-Server	ausgewogen CPU und RAM	M-Familie
Bildkonvertierung	hohe CPU-Last	C-Familie
großer In-Memory-Cache	viel RAM	R-Familie
Machine Learning mit GPU	Beschleuniger	G- oder P-Familie

Die Familie beschreibt den Schwerpunkt. Die Größe beschreibt, wie viel Leistung du innerhalb dieser Familie mietest.

m7i.1large ist also nicht nur ein zufälliger Produktname, sondern enthält Familie, Generation, Variante und Größe.

Beispiel: m6i.1large

- m: Familie
- 6: Generation
- i: Plattform- oder Prozessorvariante
- 1large: Größe innerhalb der Familie

Familie	Typische Verwendung
T	kostengünstig, burstfähig, variable leichte Last

Familie	Typische Verwendung
M	ausgewogenes Verhältnis aus CPU und RAM
C	CPU-intensive Verarbeitung
R / X	hoher Arbeitsspeicherbedarf
I / D	hoher lokaler Storage-Durchsatz
G / P	GPU, Grafik, Machine Learning

Neuere Generationen liefern häufig ein besseres Preis-Leistungs-Verhältnis. Die Auswahl muss trotzdem zu Architektur, Software und Prozessorplattform passen.

6.3 Verbindungsmethoden

- **SSH:** Linux, normalerweise Port 22 und privater Schlüssel.
- **RDP:** Windows, normalerweise Port 3389.
- **EC2 Instance Connect:** kurzlebige SSH-Schlüssel über AWS-API oder Browser.
- **EC2 Instance Connect Endpoint:** Zugriff auf private Instanzen ohne öffentliches SSH.
- **Systems Manager Session Manager:** verwalteter und protokollierbarer Zugriff ohne offenen Inbound-Port.

Für öffentlichen SSH-Zugriff müssen öffentliche IP, Route zum Internet Gateway, Security Group und NACL zusammenpassen. Port 22 sollte nie dauerhaft für 0.0.0.0/0 geöffnet sein.

6.4 User Data und Launch Templates

User Data führt beim ersten Start ein Initialisierungsskript aus. Typische Aufgaben:

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl enable --now httpd
echo "<h1>Server bereit</h1>" > /var/www/html/index.html
```

Fehler findest du in den Cloud-Init-Logs. Launch Templates speichern AMI, Instanztyp, Netzwerk, Metadatenoptionen, Rollen und User Data als wiederverwendbare Startdefinition. Sie sind die Grundlage für konsistente Auto-Scaling-Umgebungen.

6.5 AMI

Ein AMI ist eine Startvorlage mit Betriebssystem, Software und Metadaten. Ein Golden Image wird vorbereitet, getestet, versioniert und anschließend für identische Instanzen verwendet.

Vorteile:

- schneller Start
- weniger Bootstrapping-Fehler
- identische Sicherheits- und Softwarebasis
- besser kontrollierbare Skalierung

AMIs sollten Tags, Versionsnamen und einen automatisierten Lebenszyklus besitzen. Alte oder ungepatchte Images gehören ausgemustert.

7. EC2-Netzwerk, Metadaten und Platzierung

Praxis: Wie ein Paket den Server findet

Die ENI ist die virtuelle Netzwerkkarte der EC2-Instanz. An ihr hängen unter anderem private IP-Adressen und Security Groups.

```
Browser -> öffentliche Adresse / Load Balancer -> Route -> Subnetz -> ENI -> EC2
```

Die private IP bleibt normalerweise an der ENI erhalten. Eine automatisch zugewiesene öffentliche IPv4-Adresse kann sich nach Stop und Start ändern. Eine Elastic IP ist eine bewusst reservierte statische öffentliche Adresse.

Warum nicht jeder Server eine Elastic IP braucht

In einer sauberen Webarchitektur sind die App-Server privat. Nur der öffentliche Load Balancer ist von außen erreichbar. Dadurch müssen weder IP-Adressen der App-Server veröffentlicht noch bei Skalierung gepflegt werden.

IMDS in einer echten Anwendung

Eine EC2-Anwendung mit IAM Role benötigt temporäre Credentials. Sie liest diese über den Instance Metadata Service. Deshalb ist IMDS sicherheitsrelevant. IMDSv2 verlangt einen Session-Token und erschwert den Diebstahl dieser Credentials über bestimmte Angriffswege.

Die Anwendung muss die Zugangsdaten dadurch nicht in einer Konfigurationsdatei speichern. AWS SDKs holen und erneuern die temporären Credentials automatisch.

7.1 Private, öffentliche und Elastic IP

- **Private IPv4:** Kommunikation innerhalb der VPC; primäre Adresse bleibt normalerweise über Stop/Start erhalten.
- **Automatisch zugewiesene öffentliche IPv4:** internetfähig, kann sich nach Stop/Start ändern.
- **Elastic IP:** statische öffentliche IPv4, die neu zugeordnet werden kann.

Eine stabile öffentliche Quell-IP wird häufig über eine Elastic IP am NAT Gateway realisiert, nicht durch eine Elastic IP an jeder privaten Instanz.

7.2 ENI

Eine Elastic Network Interface ist eine virtuelle Netzwerkkarte. Sie enthält unter anderem private IP-Adressen, MAC-Adresse und Security-Group-Zuordnungen. ENIs ermöglichen zusätzliche Netzwerkkarten, getrennte Datenpfade und bestimmte Failover-Muster innerhalb derselben AZ.

7.3 Instance Metadata Service

Die Adresse 169.254.169.254 stellt Metadaten und temporäre Rollenzugangsdaten bereit. IMDSv2 fordert ein Session-Token und schützt besser gegen bestimmte SSRF-Angriffe. Sichere Launch Templates erzwingen IMDSv2.

7.4 Placement Groups

Drei verschiedene Anforderungen

- Ein Rechencluster tauscht sehr viele Daten zwischen Knoten aus: **Cluster Placement Group** für geringe Latenz.
- Drei besonders wichtige Instanzen sollen möglichst nicht auf derselben Hardware liegen: **Spread Placement Group**.
- Ein großes Kafka- oder Hadoop-Cluster soll in getrennte Ausfallbereiche aufgeteilt werden: **Partition Placement Group**.

Placement Groups ersetzen keine Multi-AZ-Architektur. Sie beeinflussen die Platzierung innerhalb der unterstützten Infrastruktur und lösen ein spezifisches Performance- oder Isolationsproblem.

Typ	Ziel	Typischer Einsatz
Cluster	minimale Latenz und hoher Durchsatz zwischen Instanzen	HPC, eng gekoppelte Berechnung
Spread	möglichst getrennte Hardware für wenige kritische Instanzen	Controller, kleine HA-Gruppe
Partition	große verteilte Systeme über getrennte Fehlerdomänen	Kafka, Hadoop, Cassandra

Cluster priorisiert Performance, Spread Isolation und Partition skalierbare Fehlerdomänen.

7.5 Zustände

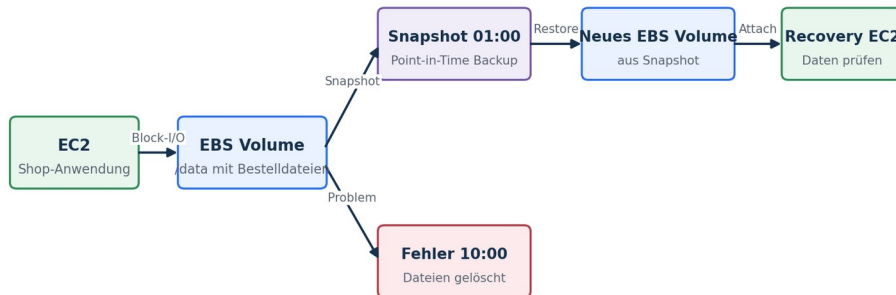
Zustand	Rechenleistung	RAM	EBS	Instance Store
Running	aktiv	aktiv	verfügbar	verfügbar
Stopped	aus	verloren	bleibt erhalten	nicht als dauerhaft betrachten
Hibernated	aus	auf EBS gesichert	bleibt erhalten	nicht dauerhaft
Terminated	gelöscht	verloren	Root meist nach Einstellung gelöscht	verloren

Hibernate ist für Workloads sinnvoll, deren In-Memory-Zustand erhalten und schnell wieder aufgenommen werden soll. Voraussetzung sind unterstützte Instanz, Betriebssystem und ein ausreichend großes verschlüsseltes EBS-Root-Volume.

8. EC2-Speicher: EBS und Instance Store

EBS, Snapshot und Wiederherstellung

Das Volume ist die laufende Festplatte. Der Snapshot ist ein zeitspunktbezogener Sicherungsstand.



Wichtig: Ein Snapshot ändert das laufende Volume nicht. Für die Wiederherstellung wird meist ein neues Volume erstellt und anschließend angehängt oder als Ersatz verwendet.

Praxisvisualisierung

Praxis: Shop-Daten, Cache und Backups

Der Shop benötigt drei unterschiedliche Arten von Daten:

27. Betriebssystem und Anwendungsdateien auf einem EBS-Root-Volume.
28. Dauerhafte Daten, die nach Stop und Start noch vorhanden sein müssen, ebenfalls auf EBS oder einem Managed Service.
29. Kurzlebige Cache- oder Zwischendaten, die neu erzeugt werden können, eventuell auf Instance Store.

EBS: „Diese Daten muss ich morgen noch haben.“

Instance Store: „Diese Daten machen die aktuelle Verarbeitung schneller, sind aber ersetzbar.“

Snapshot-Wiederherstellung

Um 01:00 Uhr wird ein Snapshot des Datenvolumes erstellt. Um 10:00 Uhr löscht ein Fehler wichtige Dateien.

30. Aus dem Snapshot wird ein neues EBS-Volume erstellt.
31. Das neue Volume wird an eine Recovery-Instanz angehängt.
32. Die Daten werden geprüft und zurückkopiert oder das Volume wird als Ersatz verwendet.

Der Snapshot ist kein zweites laufendes Volume. Er ist ein Sicherungsstand, aus dem neue Volumes erzeugt werden können.

Resize in zwei Ebenen

Wenn du ein EBS-Volume von 20 auf 40 GiB vergrößerst, sieht zunächst AWS die größere virtuelle Festplatte. Danach muss häufig auch die Partition oder das Dateisystem im Betriebssystem erweitert werden. Nur die AWS-Konsole zu ändern reicht daher nicht immer aus.

8.1 Grundunterschied

Eigenschaft	EBS	Instance Store
Art	netzwerkgebundener Blockspeicher	lokale physische Platte des Hosts
Dauerhaftigkeit	persistent	temporär
Stop/Start	Daten bleiben	Daten können verloren gehen
Snapshots	unterstützt	kein nativer EBS-Snapshot
Einsatz	Root-Volume, Dateisysteme, Datenbanken	Cache, Scratch, Zwischenverarbeitung

Merksatz: EBS ist die dauerhafte virtuelle Festplatte. Instance Store ist sehr schneller Arbeitsbereich, aber kein System of Record.

8.2 Wichtige EBS-Typen

- **gp3:** Standardempfehlung für viele Workloads; Größe, IOPS und Durchsatz können weitgehend getrennt gewählt werden.
- **io2:** für besonders hohe und konsistente IOPS, niedrige Latenz und kritische Transaktionssysteme.

- HDD-Typen eignen sich eher für sequenziellen Durchsatz als für viele zufällige I/O-Operationen.

8.3 Verschlüsselung, Snapshots und Änderung

EBS-Verschlüsselung nutzt KMS und schützt Volume, Snapshots und den Datenpfad zwischen Host und Volume. Ein EBS-Volume ist an eine AZ gebunden und kann nur an Instanzen derselben AZ angehängt werden.

Snapshots:

- sind Point-in-Time-Sicherungen
- speichern nach dem ersten Snapshot nur geänderte Blöcke
- können neue Volumes erzeugen
- können für Disaster Recovery in andere Regionen kopiert werden

Bei einer Größenänderung reicht die AWS-Änderung allein nicht. Danach müssen je nach Betriebssystem Partition und Dateisystem erweitert werden.

9. EC2-Kosten und Tenancy

Praxis: Baseline, Spitzenlast und Batch-Jobs

Der Shop hat eine Grundlast, die jeden Tag vorhanden ist. Zusätzlich gibt es unregelmäßige Spitzen und einen nächtlichen Job zur Bildoptimierung.

Grundlast der App-Server	-> Savings Plan oder passende Reservierung nach Messung
Unvorhersehbare Spitzen	-> On-Demand
Unterbrechbarer Bild-Job	-> Spot
Garantierter Start in einer AZ	-> Capacity Reservation
Lizenz an physische CPUs gebunden	-> Dedicated Host

Die günstigste Option ist nur dann die beste, wenn sie zur technischen Anforderung passt. Ein einzelner nicht unterbrechbarer Datenbankserver auf Spot wäre zwar billig, aber nicht belastbar.

Erst messen, dann binden

Ein neues Projekt beginnt häufig mit On-Demand. CloudWatch- und Compute-Optimizer-Daten zeigen später, welche Instanzgröße wirklich benötigt wird und welche Grundlast stabil ist. Erst dann lohnt sich eine längerfristige Verpflichtung.

Rightsizing ist oft die erste Einsparung: Eine halb ausgelastete kleinere Instanz kann sinnvoller sein als ein Rabatt auf eine deutlich zu große Instanz.

9.1 Kaufmodelle

Modell	Bindung	Unterbrechung	Geeignet für
On-Demand	keine	nein	kurze, unbekannte oder neue Workloads
Spot	keine	möglich	Batch, CI, Rendering, fehlertolerante Systeme
Reserved Instances	1 oder 3 Jahre	nein	stabile EC2-Nutzung
Savings Plans	1 oder 3 Jahre Ausgabenbindung	nein	planbare Grundlast mit mehr Flexibilität
Capacity Reservation	reservierte Kapazität in einer AZ	nein	garantierter Start bei kritischen Ereignissen
Dedicated Host	physischer Host	nein	BYOL, Compliance und Host-Sichtbarkeit

Ein gutes Kostenmodell kombiniert oft mehrere Optionen: Savings Plans oder RIs für die Grundlast, On-Demand für variable Last und Spot für unterbrechbare Verarbeitung.

9.2 Rightsizing

Der günstigste Rabatt hilft wenig, wenn die Instanz dauerhaft viel zu groß ist. CloudWatch-Metriken, Compute Optimizer und Kostenberichte helfen beim Rightsizing. Erst Nutzung messen, dann sinnvoll binden.

9.3 Tenancy

- **Shared:** Standard, kostengünstig, Hardware wird sicher mit anderen Kunden geteilt.
- **Dedicated Instance:** Instanzen laufen auf Hardware, die nur deinem Konto zugeordnet ist, ohne volle Host-Sicht.
- **Dedicated Host:** vollständiger physischer Host mit Lizenz- und Platzierungskontrolle.

10. VPC, CIDR und Subnetze

Praxis: Ein vollständiger Adressplan

Für den Shop wird das VPC-Netz `10.0.0.0/16` reserviert. Daraus entstehen vier Subnetze:

```
VPC 10.0.0.0/16
|
+-- AZ A
|   +-- Public-A 10.0.1.0/24
|   +-- Private-A 10.0.11.0/24
|
+-- AZ B
|   +-- Public-B 10.0.2.0/24
|   +-- Private-B 10.0.12.0/24
```

Die öffentlichen Subnetze enthalten den Load Balancer und bei Bedarf NAT Gateways. Die privaten Subnetze enthalten die App-Server und Datenbankkomponenten.

Was ein Subnetz wirklich öffentlich macht

Ein Subnetz wird nicht durch seinen Namen öffentlich. Entscheidend ist die Route Table. Für IPv4 enthält ein öffentliches Subnetz typischerweise:

```
10.0.0.0/16 -> local
0.0.0.0/0    -> Internet Gateway
```

Damit eine EC2-Instanz direkt aus dem Internet erreichbar ist, braucht sie zusätzlich eine öffentliche Adresse und passende Security-Regeln. Eine Route zum IGW allein reicht nicht.

Warum CIDR-Planung später wichtig wird

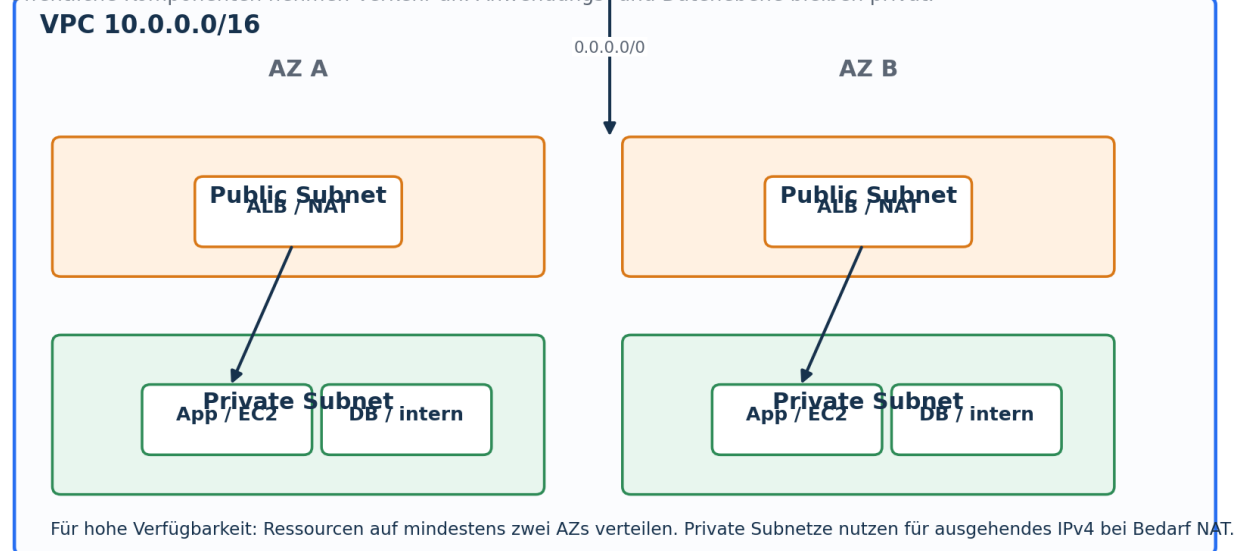
Wenn später ein zweites VPC per Peering angebunden werden soll, dürfen sich die Adressbereiche nicht überschneiden. Ein unüberlegt gewähltes `10.0.0.0/16` in jedem VPC führt deshalb später zu Routingproblemen.

10.1 VPC

Eine VPC ist dein logisch isoliertes Netzwerk in AWS. Sie enthält CIDR-Bereiche, Subnetze, Route Tables, Gateways, Security Groups, NACLs und Netzwerkinterfaces.

Referenz-VPC über zwei Availability Zones

Öffentliche Komponenten nehmen Verkehr an. Anwendungs- und Datenebene bleiben privat.



Referenz-VPC über zwei Availability Zones

10.2 CIDR

CIDR bestimmt die Größe eines Netzbereichs:

CIDR	Adressen gesamt	In AWS-Subnetz nutzbar
/16	65.536	65.531

CIDR	Adressen gesamt	In AWS-Subnetz nutzbar
/24	256	251
/26	64	59
/28	16	11

AWS reserviert in jedem Subnetz fünf Adressen. CIDR-Bereiche sollten ausreichend groß und zwischen VPCs nicht überlappend geplant werden, besonders wenn Peering oder Transit Gateway vorgesehen ist.

10.3 Public und Private Subnet

Ein Subnetz ist nicht wegen seines Namens öffentlich, sondern wegen seiner Route Table.

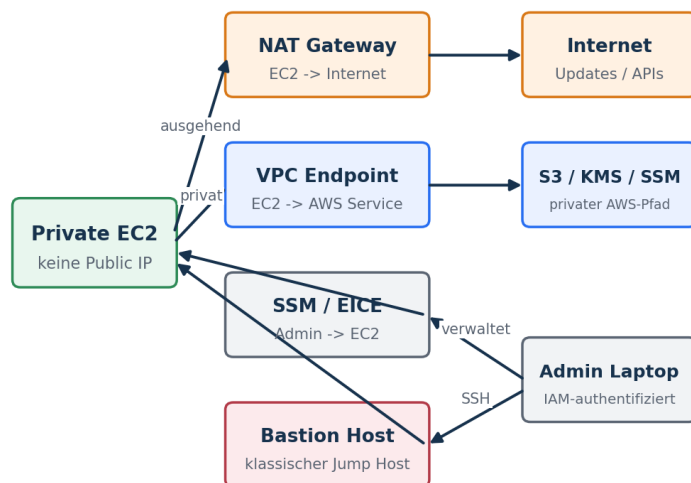
- **Public Subnet:** Default Route `0.0.0.0/0` zum Internet Gateway.
- **Private Subnet:** keine direkte Default Route zum Internet Gateway.
- Für direkte IPv4-Internetkommunikation benötigt eine Instanz zusätzlich eine öffentliche oder Elastic IP.

Eine Route Table kann mehreren Subnetzen zugeordnet werden. Ein Subnetz nutzt gleichzeitig genau eine Route Table. Die spezifischste Route gewinnt: **Longest Prefix Match**.

11. Internet, NAT und sichere Admin-Wege

Vier Wege, die oft verwechselt werden

Internet-Zugriff, Admin-Zugriff und privater AWS-Service-Zugriff sind unterschiedliche Anforderungen.



NAT

macht keine eingehenden Admin-Sitzungen möglich.

Endpoint

ersetzt NAT nur für die unterstützten AWS-Dienste.

SSM / EICE

ermöglicht Administration ohne öffentlichen SSH-Port.

Bastion

ist ein eigener Server und muss selbst gehärtet werden.

Praxisvisualisierung

Praxis: Zwei Anforderungen, zwei völlig verschiedene Wege

Fall 1: Der private App-Server lädt Updates herunter

Der Server startet die Verbindung selbst. Die private Route Table zeigt für unbekannte IPv4-Ziele auf ein NAT Gateway im öffentlichen Subnetz.

Private EC2 -> Private Route Table -> NAT Gateway -> Internet Gateway -> Repository

Antwortpakete kommen über denselben Weg zurück. Eine neue Verbindung aus dem Internet direkt zur privaten EC2-Instanz wird dadurch nicht ermöglicht.

Fall 2: Ein Administrator braucht eine Shell

Das ist keine normale ausgehende Anwendungsverbindung. Es ist ein Managementzugriff. Mögliche Lösungen sind:

- Systems Manager Session Manager
- EC2 Instance Connect Endpoint
- klassischer Bastion Host

Admin -> IAM-authentifizierter Managementpfad -> private EC2

NAT Gateway und Bastion Host werden oft verwechselt. NAT ist ein Egress-Weg für Ressourcen. Ein Bastion Host ist ein Einstiegspunkt für Menschen oder Admin-Werkzeuge.

Warum VPC Endpoints zusätzlich sinnvoll sind

Wenn die private EC2 nur S3 und Systems Manager benötigt, muss dieser Verkehr nicht über NAT ins öffentliche Servicenetz laufen. Passende VPC Endpoints bieten einen privaten AWS-Pfad und können NAT-Kosten sowie öffentliche Abhängigkeiten reduzieren.

11.1 Internet Gateway

Ein Internet Gateway wird an eine VPC angehängt. Es ermöglicht direkte IPv4- und IPv6-Kommunikation, wenn Routing, IP-Adressierung und Sicherheitsregeln passen. Das reine Anhängen macht noch kein Subnetz öffentlich.

11.2 Egress-Only Internet Gateway

Ein Egress-Only Internet Gateway ist für ausgehenden IPv6-Verkehr gedacht. Es erlaubt Verbindungen, die intern gestartet wurden, blockiert aber unerwünschte neue Verbindungen von außen.

11.3 NAT Gateway

Ein NAT Gateway befindet sich in einem Public Subnet, besitzt eine Elastic IP und leitet ausgehenden IPv4-Verkehr privater Subnetze zum Internet weiter.

Ausgehender Internetzugriff aus einem privaten Subnetz

NAT Gateway sitzt öffentlich. Die private Instanz bleibt ohne direkt erreichbare öffentliche IPv4.



Keine neue Verbindung aus dem Internet wird über NAT zur privaten Instanz weitergeleitet.

Für hohe Verfügbarkeit: häufig ein NAT Gateway pro AZ und private Routen zur NAT-Instanz derselben AZ.

NAT Gateway für private Subnetze

Für hohe Verfügbarkeit wird häufig ein NAT Gateway pro AZ verwendet, damit ein AZ-Ausfall nicht den gesamten ausgehenden Verkehr blockiert. NAT ist kein Weg für eingehende SSH-Verbindungen.

11.4 Bastion, EICE und Session Manager

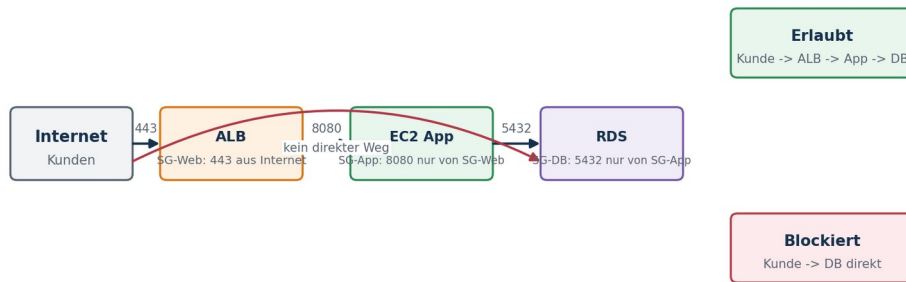
- **Bastion Host:** gehärtete EC2-Instanz als Jump Host. Funktioniert, erzeugt aber Patch-, Schlüssel- und Monitoring-Aufwand.
- **EC2 Instance Connect Endpoint:** verwalteter Weg zu privaten EC2-Instanzen, ohne öffentlichen SSH-Port.
- **Session Manager:** IAM-gesteuerter Zugang über Systems Manager, zentral protokollierbar und ohne eingehenden Port.

Merksatz: NAT ist der Paketausgang. Bastion oder Managed Access ist der Personeneingang.

12. Security Groups, NACLs und Network Firewall

Dreistufige Anwendung: Security Groups als Beziehung

Nicht jede Schicht darf direkt aus dem Internet erreichbar sein.



Sinn: Die Regeln orientieren sich an der Architektur. Ein Datenbank-Port ist nicht „sicher“, weil er ungewöhnlich ist, sondern weil nur die Anwendungsschicht ihn erreichen darf.

Praxisvisualisierung

Praxis: Web-, App- und Datenbankschicht

Eine sichere Freigabe wird nicht nach dem Prinzip „Port ist offen oder geschlossen“ geplant, sondern nach Beziehungen zwischen Schichten.

```

Internet
| TCP 443
v
ALB mit SG-Web
| TCP 8080, Quelle = SG-Web
v
EC2 mit SG-App
| TCP 5432, Quelle = SG-App
v
Datenbank mit SG-DB
  
```

Der Kunde kann Port 5432 der Datenbank nicht direkt erreichen. Nur Ressourcen mit der App-Security-Group dürfen die Datenbankschicht ansprechen.

Stateful praktisch erklärt

Der Client baut eine Verbindung zum Webserver auf. Die Security Group erlaubt den Eingang auf Port 443. Die Antwort darf automatisch zurück, obwohl dafür keine spiegelbildliche Inbound-Regel auf der Client-Seite der Security Group definiert wird.

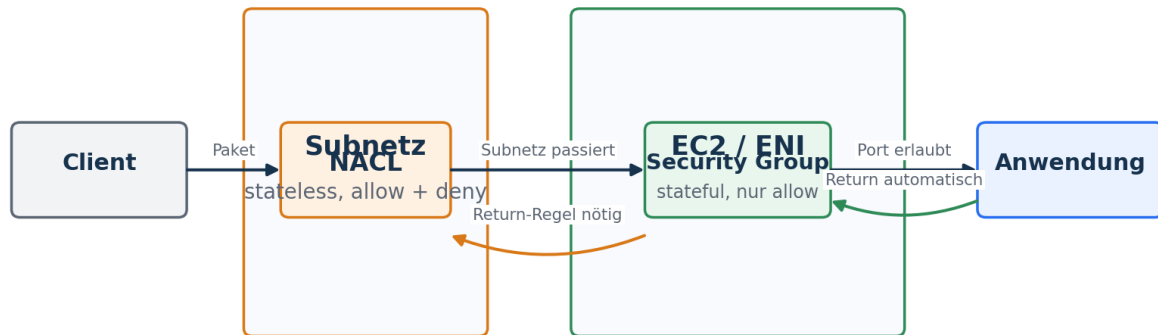
Eine NACL betrachtet jedes Paket getrennt. Deshalb müssen bei NACLs auch Rückwege und kurzlebige Clientports berücksichtigt werden.

Wann Network Firewall ins Spiel kommt

Security Groups schützen einzelne Workloads, NACLs Subnetzgrenzen. AWS Network Firewall wird eingesetzt, wenn Verkehr zentral über eine Prüfstrecke geführt und beispielsweise nach Domains, Protokollen oder Signaturen kontrolliert werden soll. Dafür muss die Route bewusst durch Firewall-Subnetze gelegt werden.

Security Group und NACL: Zwei Schutzebenen

Beide müssen Verkehr zulassen. Der Hauptunterschied ist Stateful vs. Stateless.



Merksatz: Security Group = Tür an der Instanz. NACL = Schranke am Subnetz.

Security Group und NACL

12.1 Security Group

- sitzt an ENI oder Ressource
- stateful: Rückverkehr einer erlaubten Verbindung wird automatisch zugelassen
- nur Allow-Regeln
- alle Regeln werden gemeinsam betrachtet
- kann andere Security Groups als Quelle referenzieren

Ein typisches dreistufiges Modell:

- Web-SG erlaubt 443 aus dem Internet
- App-SG erlaubt Anwendungsport nur von Web-SG
- DB-SG erlaubt Datenbankport nur von App-SG

12.2 NACL

- sitzt am Subnetz
- stateless: Hin- und Rückrichtung benötigen passende Regeln
- Allow und Deny möglich
- Regeln werden aufsteigend nach Nummer geprüft; erste passende Regel gewinnt
- Custom NACL startet ohne Freigaben

Fehlende Ephemeral-Port-Regeln sind ein häufiger Grund für Timeouts, obwohl die Security Group korrekt aussieht.

12.3 AWS Network Firewall

Network Firewall ist eine zentral verwaltete, stateful Inspektions- und Filterlösung für komplexe Ingress-, Egress- und East-West-Regeln. Sie ergänzt Security Groups und NACLs, ersetzt sie aber nicht automatisch. Der Dienst benötigt bewusstes Routing durch Firewall-Subnetze und erhöht Kosten und Komplexität.

13. Private AWS-Zugriffe, Peering und Hybrid Networking

Praxis: Welche Verbindung passt zu welchem Problem?

Private EC2 greift auf S3 zu

Für S3 eignet sich ein Gateway Endpoint. Die Route Table erkennt S3-Zielpräfixe und leitet sie über den Endpoint. Der Verkehr benötigt dafür kein NAT Gateway.

Private EC2 ruft Secrets Manager auf

Viele andere AWS-Dienste werden über Interface Endpoints angebunden. Im Subnetz entsteht eine ENI mit privater IP. Private DNS kann den normalen Dienstenamen auf diese private Adresse auflösen.

Zwei VPCs müssen direkt miteinander sprechen

Für wenige, klar definierte Verbindungen eignet sich VPC Peering. Beide Seiten benötigen nicht überlappende CIDRs, passende Routen und Sicherheitsregeln.

VPC Shop 10.0.0.0/16 <--- Peering ---> VPC Analytics 10.20.0.0/16

Peering ist nicht transitiv. Wenn Shop mit Analytics und Analytics mit Security verbunden ist, entsteht daraus nicht automatisch ein Weg von Shop zu Security.

Viele VPCs und ein Rechenzentrum

Bei vielen VPCs wird ein Peering-Netz unübersichtlich. Transit Gateway bündelt die Verbindungen als Hub. Das Rechenzentrum kann über Site-to-Site VPN oder Direct Connect angebunden werden.

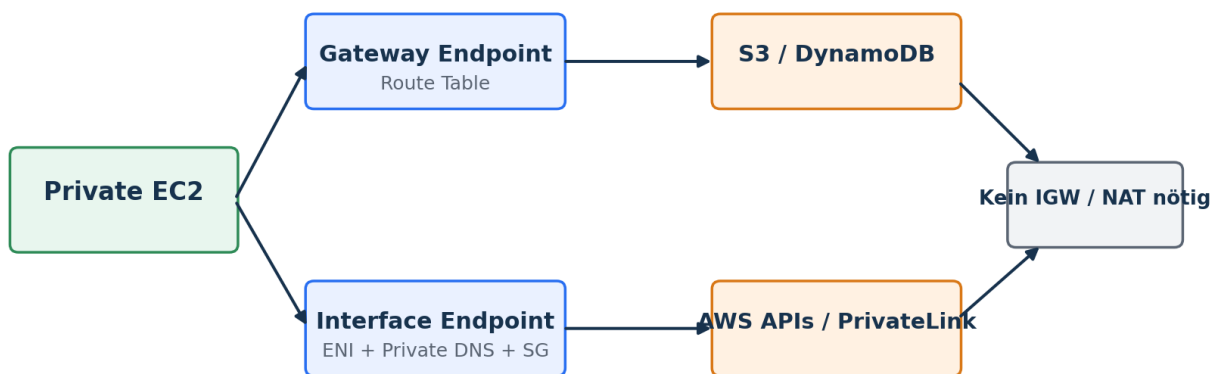
- VPN: schneller einzurichten, läuft über das Internet, gut als Einstieg oder Backup.
- Direct Connect: dedizierte Verbindung, planbarer, aber teurer und aufwendiger.

Auswahlregel: Endpoint verbindet ein VPC privat mit einem Dienst. Peering verbindet zwei VPCs. Transit Gateway verbindet viele Netze. VPN oder Direct Connect verbindet AWS mit On-Premises.

13.1 VPC Endpoints und PrivateLink

VPC Endpoints: AWS-Dienste privat erreichen

Gateway Endpoints arbeiten über Routen. Interface Endpoints sind ENIs mit privaten IPs und Security Groups.



Gateway und Interface Endpoints

Typ	Umsetzung	Dienste	Kostenmodell
Gateway Endpoint	Route-Table-Ziel	S3 und DynamoDB	kein üblicher stündlicher Endpoint-Preis
Interface Endpoint	ENI mit privater IP	viele AWS- und PrivateLink-Dienste	stündlich plus Datenverarbeitung

Endpoint Policies begrenzen, welche Aktionen über den Endpoint möglich sind. Bei Interface Endpoints müssen DNS, Security Groups und Subnetzplatzierung stimmen.

13.2 VPC Peering

Peering verbindet zwei VPCs direkt über private IPs. Voraussetzungen:

- nicht überlappende CIDRs
- Routen auf beiden Seiten
- passende Security Groups und NACLs
- Akzeptieren der Verbindung bei Cross-Account

Peering ist **nicht transitiv**. A-B und B-C ergeben keine Verbindung A-C. Bei vielen VPCs entsteht ein schwer wartbares Mesh.

13.3 Transit Gateway

Transit Gateway ist ein regionaler Routing-Hub für viele VPCs und On-Premises-Netze. Er vereinfacht Hub-and-Spoke-Topologien und transitive Verbindungen, benötigt aber sorgfältige Route Tables, Segmentierung und Kostenkontrolle.

13.4 VPN und Direct Connect

Verbindung	Vorteil	Nachteil
Site-to-Site VPN	schnell eingerichtet, niedriger Einstieg, gut als Backup	Internetabhängigkeit, schwankendere Latenz und Bandbreite
Direct Connect	privater dedizierter Pfad, planbarere Leistung	längere Bereitstellung, höhere Fixkosten

In Enterprise-Architekturen wird Direct Connect häufig durch VPN als Ausweichpfad ergänzt.

14. VPC Flow Logs und CloudTrail

Praxis: Die Anwendung erreicht die Datenbank nicht

Der App-Server meldet einen Timeout beim Verbindungsaufbau zur Datenbank.

Was Flow Logs beantworten können

- Welche Quell- und Ziel-IP waren beteiligt?
- Welcher Port wurde verwendet?
- Wurde der Verkehr akzeptiert oder abgelehnt?
- Welche ENI war betroffen?

Ein REJECT kann auf eine NACL- oder Security-Konfiguration hinweisen. Ein ACCEPT bedeutet nur, dass der Netzwerkpfad an dieser Stelle erlaubt war. Die Anwendung kann trotzdem nicht lauschen oder intern einen Fehler haben.

Was CloudTrail stattdessen beantwortet

CloudTrail zeigt Verwaltungs- und API-Ereignisse. Beispielsweise kann er zeigen, dass kurz vor dem Fehler eine Security-Group-Regel geändert wurde.

```
Flow Logs: „Paket von 10.0.11.20 zu 10.0.21.15:5432 wurde abgelehnt.“
CloudTrail: „User X hat um 14:03 eine SG-Regel gelöscht.“
App-Log: „Connection timeout zur Datenbank.“
```

Die drei Datenquellen ergänzen sich. Observability bedeutet nicht, möglichst viele Logs zu sammeln, sondern die richtigen Signale miteinander zu verbinden.

VPC Flow Logs zeichnen Metadaten zu Netzwerkverkehr auf: Quell- und Ziel-IP, Ports, Protokoll sowie ACCEPT oder REJECT. Sie enthalten keine vollständigen Paketdaten.

Mögliche Ziele:

- CloudWatch Logs für zeitnahe Suche und Alarme
- S3 für günstige Langzeitaufbewahrung und Athena-Auswertung
- Streaming-Pipeline für SIEM oder OpenSearch

Unterschied:

- **Flow Logs:** Wer kommunizierte mit wem über welchen Netzwerkpfad?
- **CloudTrail:** Wer rief welche AWS-API auf und änderte die Konfiguration?

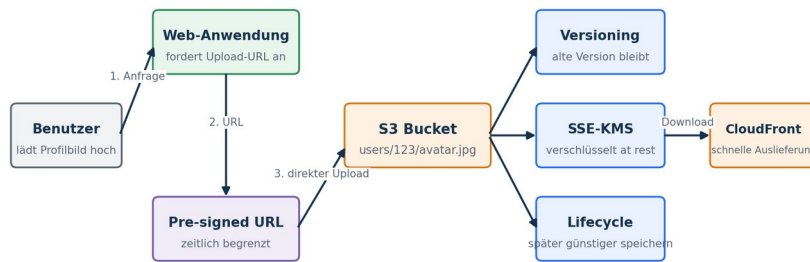
Fehlersuche bei einem Timeout:

33. Zeitfenster bestimmen.
34. Quell-IP, Ziel-IP und Port filtern.
35. ACCEPT oder REJECT prüfen.
36. Route Table, NACL und Security Group vergleichen.
37. Danach Anwendungs- und Betriebssystemlogs prüfen.

15. Amazon S3: Objektspeicher

S3 in einer echten Anwendung

S3 speichert Objekte. Die Anwendung verwaltet nur Metadaten und Berechtigungen.



Warum sinnvoll? Die Datei muss nicht durch den EC2-Server „durchgeschoben“ werden. S3 übernimmt dauerhafte Speicherung, während die Anwendung kontrolliert, wer welche Datei verwenden darf.

Praxisvisualisierung

Praxis: Produktbilder und Benutzer-Uploads

Ein Produktbild wird nicht sinnvoll als Datei auf jedem einzelnen EC2-Server gespeichert. Bei Auto Scaling könnten neue Server das Bild nicht besitzen, und beim Löschen einer Instanz wäre es weg.

Stattdessen liegt das Bild als Objekt in S3:

```

Bucket: shop-products
Key:    products/4711/main.jpg
Object: eigentliche Bilddaten + Metadaten
  
```

Die Datenbank speichert nur Informationen wie Produktnummer und Objektpfad. Die Anwendung liest oder erzeugt eine URL für das S3-Objekt.

Direkter Upload mit einer Pre-signed URL

38. Der Benutzer fragt die Webanwendung nach einer Upload-Möglichkeit.
39. Die Anwendung prüft, ob der Benutzer berechtigt ist.
40. Die Anwendung erzeugt eine zeitlich begrenzte Pre-signed URL.
41. Der Browser lädt die Datei direkt nach S3.
42. Die Anwendung speichert den Objektpfad in der Datenbank.

Dadurch muss eine große Datei nicht erst durch den App-Server transportiert werden.

Versioning, Object Lock und Lifecycle lösen verschiedene Probleme

- **Versioning:** Eine ersetzte oder gelöschte Datei kann aus einer älteren Version wiederhergestellt werden.
- **Object Lock:** Ein Objekt darf für eine festgelegte Zeit nicht überschrieben oder gelöscht werden.
- **Lifecycle:** Alte Objekte werden automatisch in günstigere Klassen verschoben oder nach Regeln entfernt.
- **Replication:** Objekte werden in einen anderen Bucket, bei CRR auch in eine andere Region, kopiert.

Lifecycle ist kein Backup und Replication ist keine automatische Löschsperre. Jeder Mechanismus hat einen anderen Zweck.

15.1 Bucket und Objekt

S3 ist Objektspeicher und keine klassische Festplatte. Ein Objekt besteht aus Daten, Key und Metadaten. Ein Bucket bildet den Namens- und Policy-Rahmen.

Typische Nutzung:

- Dateien und Medien
- Backups
- Logs und Datenarchive
- statische Webseiten
- Terraform State
- Data-Lake-Daten

S3 vs. EBS: S3 wird über Objekt-APIs angesprochen. EBS ist Blockspeicher für ein Dateisystem oder eine Datenbank auf einer Instanz.

15.2 Zugriffsschutz

- Block Public Access als sichere Grundeinstellung
- IAM Policies für Berechtigungen eines Principals
- Bucket Policies für Regeln direkt am Bucket und Cross-Account-Zugriff
- ACLs sind ein älterer, feingranularer Mechanismus und werden möglichst vermieden
- Pre-Signed URLs geben zeitlich begrenzten Zugriff auf ein einzelnes Objekt

15.3 Versionierung und Schutz vor Löschen

Versionierung bewahrt ältere Objektversionen. Ein normales Löschen erzeugt oft einen Delete Marker statt die alten Daten sofort zu entfernen. MFA Delete erschwert kritische Versionierungs- und Löschaktionen.

Object Lock implementiert WORM-Aufbewahrung:

- **Governance Mode:** privilegierte Benutzer können unter definierten Bedingungen übersteuern.
- **Compliance Mode:** während der Aufbewahrungszeit ist selbst administratives Löschen nicht möglich.

15.4 Verschlüsselung

Praktische Auswahl

- **SSE-S3:** einfache serverseitige Verschlüsselung, Schlüsselverwaltung durch S3.
- **SSE-KMS:** zusätzliche Kontrolle über Schlüsselrichtlinien und nachvollziehbare KMS-Nutzung.
- **SSE-C:** der Kunde liefert den Schlüssel bei jeder Anfrage; höherer eigener Betriebsaufwand.

Für den Shop können normale Produktbilder mit SSE-S3 ausreichen. Für besonders sensible Dokumente kann SSE-KMS sinnvoll sein, weil Schlüsselzugriff zusätzlich kontrolliert und protokolliert werden kann.

Verschlüsselung beantwortet die Frage, wie Daten geschützt gespeichert werden. Sie ersetzt keine Zugriffspolicy. Ein verschlüsseltes Objekt kann trotzdem zu breit freigegeben sein.

Methode	Schlüsselverwaltung	Besonderheit
SSE-S3	S3 verwaltet Schlüssel	einfache serverseitige Verschlüsselung
SSE-KMS	AWS KMS	Key Policies, Audit und stärkere Kontrolle
SSE-C	Kunde liefert Schlüssel je Anfrage	Kunde trägt Schlüsselverantwortung
Client-side	vor Upload verschlüsselt	AWS sieht nur Ciphertext

SSE-KMS benötigt zusätzlich zu S3-Rechten passende KMS-Berechtigungen.

15.5 Lifecycle und Replikation

Lifecycle-Regeln verschieben Objekte abhängig von Alter, Präfix oder Tags in günstigere Storage Classes oder löschen sie nach einer Frist. Dabei müssen Mindestaufbewahrung und Abrufkosten berücksichtigt werden.

- **SRR:** Replikation in derselben Region, etwa für getrennte Konten oder Compliance.
- **CRR:** Replikation in eine andere Region, etwa für DR oder Datenstandort.

Replikation ist kein Ersatz für Versionierung und kein automatischer Schutz gegen jede fehlerhafte Anwendung. Das Ziel muss passend abgesichert sein.

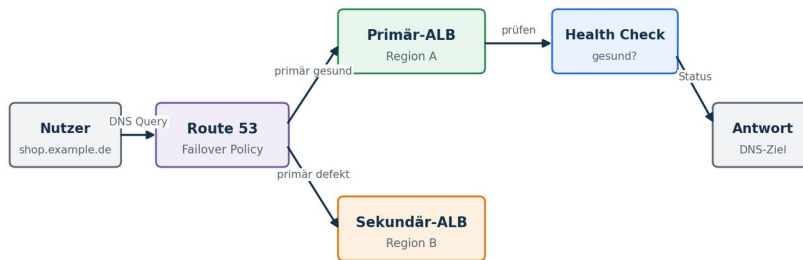
15.6 Performance und Betrieb

- Multipart Upload zerlegt große Dateien und ermöglicht parallele Übertragung.
- Transfer Acceleration nutzt das globale Edge-Netz für weit entfernte Uploads.
- Batch Operations führt Aktionen über sehr viele Objekte aus.
- Storage Lens zeigt Nutzung und Optimierungspotenzial organisationsweit.
- Access Logs und CloudTrail Data Events helfen bei der Nachvollziehbarkeit.

16. Route 53 und DNS

Route 53 Failover: Primärziel und Ersatzweg

DNS entscheidet nicht, ob eine Anwendung gesund ist. Ein Health Check liefert dafür das Signal.



Praxis: Nach einem Ausfall wechseln neue DNS-Auflösungen zum Sekundärziel. Bereits bestehende Verbindungen werden dadurch nicht automatisch verschoben; TTL und Client-Caches spielen mit hinein.

Praxisvisualisierung

Praxis: Vom Domainnamen bis zum Load Balancer

Der Browser kennt zunächst nur `shop.example.de`.

43. Ein DNS-Resolver fragt nach dem zuständigen Eintrag.
44. Route 53 antwortet mit dem Ziel des Records, häufig einem Alias auf CloudFront oder einen Load Balancer.
45. Der Browser verbindet sich mit dem aufgelösten Ziel.
46. Ab dann übernehmen Netzwerk, TLS, Load Balancer und Anwendung.

Route 53 transportiert keine Produktseite. DNS liefert nur die Information, **wohin** sich der Client verbinden soll.

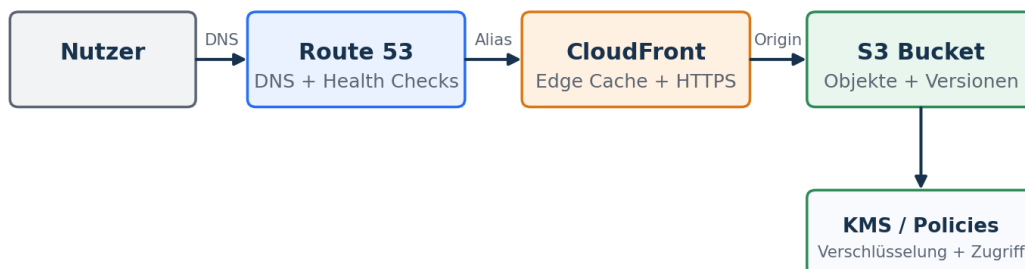
Auswahl der Routing Policy an konkreten Zielen

- **Weighted:** 10 Prozent der Nutzer sollen die neue Version testen.
- **Latency:** Nutzer sollen zur Region mit der voraussichtlich geringsten Netzwerklatenz.
- **Failover:** Primärziel verwenden, bei fehlgeschlagenem Health Check auf Sekundärziel wechseln.
- **Geolocation:** Antworten anhand des geografischen Standorts steuern.

Bei Failover spielen TTL und Caches eine Rolle. Ein DNS-Wechsel ist daher kein identischer Ersatz für einen Load Balancer, der laufende Anfragen innerhalb einer Region verteilt.

Statische Inhalte: Route 53, CloudFront und S3

S3 speichert Objekte. CloudFront verteilt sie weltweit. Route 53 löst den Namen auf.



S3 Website Endpoint allein ist einfach, aber CloudFront ergänzt TLS, Caching, Zugriffsschutz und globale Performance.

Route 53, CloudFront und S3

16.1 Hosted Zones

- **Public Hosted Zone:** DNS-Namen sind öffentlich im Internet auflösbar.

- **Private Hosted Zone:** Namen sind nur in verknüpften VPCs auflösbar.

16.2 Wichtige Records

Record	Zweck
A	Name zu IPv4
AAAA	Name zu IPv6
CNAME	Name zu anderem Namen; nicht für Zone Apex
Alias	AWS-spezifische Zuordnung zu ALB, CloudFront, S3 und anderen Zielen
MX	Mailserver
TXT	Verifikation und beliebige Textinformationen

TTL bestimmt, wie lange Resolver eine Antwort zwischenspeichern. Kleine TTLs erlauben schnellere Änderungen, erzeugen aber mehr DNS-Anfragen.

16.3 Routing Policies

- **Simple:** ein oder mehrere normale Ziele ohne besondere Steuerung.
- **Weighted:** prozentuale Verteilung, ideal für Blue/Green oder schrittweisen Rollout.
- **Latency:** Ziel mit voraussichtlich geringster Netzwerklatenz.
- **Failover:** Primary/Secondary mit Health Check.
- **Geolocation:** Routing nach geografischem Standort.
- **Multivalue:** mehrere gesunde Antworten, einfache Verteilung.

Ein Health Check prüft die Erreichbarkeit eines Endpunkts. Er ersetzt keine vollständige Anwendungsbeobachtung, ermöglicht aber DNS-basiertes Failover.

17. Terraform und Infrastructure as Code

Praxis: Warum der Shop nicht dauerhaft per Klick aufgebaut wird

Die erste Testumgebung lässt sich noch per Konsole erstellen. Später werden Development, Test und Production benötigt. Werden alle Umgebungen manuell geklickt, entstehen Abweichungen:

```
Development: Port 22 offen, gp3, zwei Subnetze
Test:        Port 22 geschlossen, gp2, ein Subnetz
Production:  andere Tags, alte AMI, fehlendes Logging
```

Terraform beschreibt den gewünschten Zustand als Code. Dieselbe Grundstruktur kann mit unterschiedlichen Variablen wiederverwendet werden.

```
resource "aws_instance" "shop_app" {
  ami           = var.ami_id
  instance_type = var.instance_type
  subnet_id     = var.private_subnet_id

  tags = {
    Name        = "shop-app"
    Environment = var.environment
  }
}
```

Der Code ersetzt nicht das Architekturwissen. Terraform erstellt genau das, was beschrieben wurde - auch eine unsichere oder teure Architektur. IaC macht Entscheidungen wiederholbar und prüfbar, aber nicht automatisch richtig.

17.1 Warum IaC?

Infrastructure as Code beschreibt Infrastruktur in versionierten Dateien. Das bringt:

- Wiederholbarkeit
- Code Review
- nachvollziehbare Änderungen
- identische Dev-, Test- und Produktionsmuster
- weniger manuelle Konfigurationsabweichung
- automatisierte CI/CD-Abläufe

Terraform ist deklarativ: Du beschreibst den Zielzustand. Terraform baut daraus einen Abhängigkeitsgraphen und plant die notwendigen API-Aufrufe.

17.2 Wichtige Blocktypen

```
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_s3_bucket" "logs" {
  bucket = var.bucket_name
}

data "aws_caller_identity" "current" {}

output "account_id" {
  value = data.aws_caller_identity.current.account_id
}
```

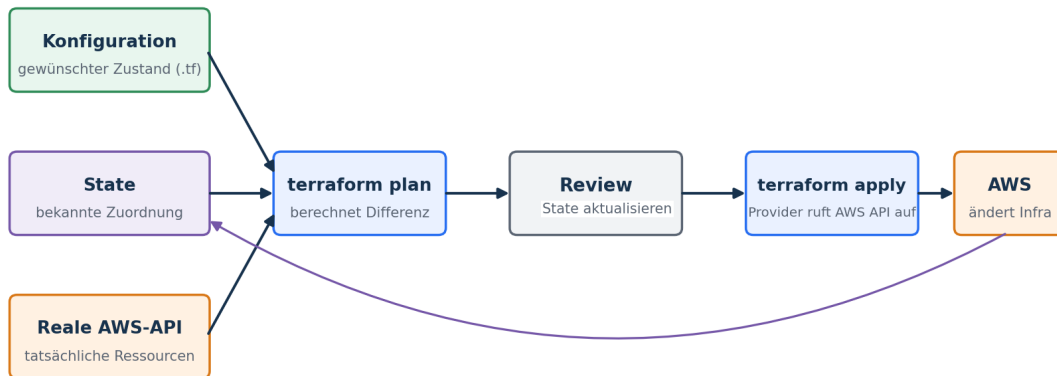
- **terraform**: CLI- und Provider-Anforderungen
- **provider**: Zielplattform und Konfiguration
- **resource**: Ressource erstellen oder verwalten
- **data**: vorhandene Informationen nur lesen
- **variable**: Eingabe
- **locals**: interne berechnete Werte
- **output**: ausgewählte Werte ausgeben
- **module**: wiederverwendbare Konfiguration aufrufen

Die Datei `.terraform.lock.hcl` hält ausgewählte Provider-Versionen und Prüfsummen fest. Sie gehört normalerweise in die Versionskontrolle. Das Verzeichnis `.terraform` und lokale State-Dateien gehören normalerweise nicht hinein.

18. Terraform-Workflow

Terraform in der Praxis: Code, State, Provider und AWS

Terraform vergleicht drei Sichten und erzeugt daraus einen Plan.



Wichtig: Terraform „merkt“ sich nicht nur Code. Ohne State fehlt die Zuordnung zwischen `aws_instance.web` im Code und der echten Instanz-ID in AWS.

Praxisvisualisierung

Was bei jedem Befehl konkret passiert

`terraform init`

Terraform bereitet das Arbeitsverzeichnis vor. Provider und Module werden geladen, der Backend-Zugriff wird eingerichtet und die Lock-Datei berücksichtigt.

`terraform validate`

Terraform prüft, ob die Konfiguration intern gültig ist. Dabei wird nicht vollständig bewiesen, dass die Ressource in AWS erfolgreich erstellt werden kann.

`terraform plan`

Terraform liest Konfiguration, State und den bekannten realen Zustand. Daraus entsteht eine Änderungsvorschau.

```

+ create  aws_instance.shop_app
~ update  aws_security_group.app
- destroy aws_instance.old_server
  
```

`terraform apply`

Der Provider übersetzt den Plan in AWS-API-Aufrufe. Nach erfolgreichen Aktionen wird der State aktualisiert.

Warum der Plan gelesen werden muss

Ein Plan ist kein lästiger Zwischenschritt. Er ist die letzte Stelle, an der du ohne Veränderung erkennen kannst, dass beispielsweise eine Datenbank ersetzt oder ein komplettes Subnetz gelöscht werden soll.

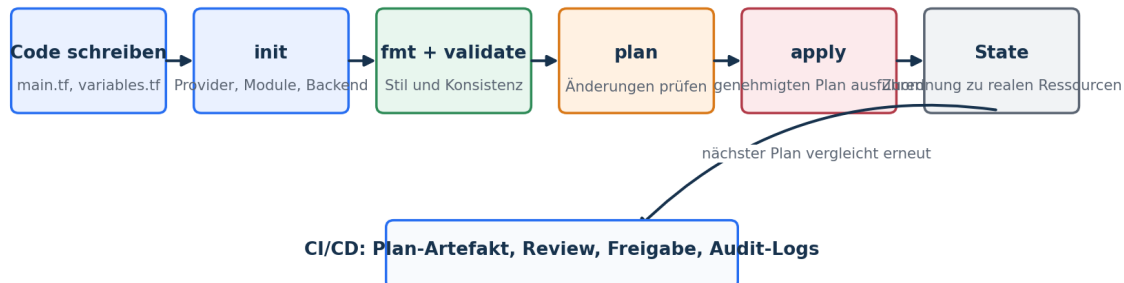
Bei einem unerwartet großen Plan zuerst prüfen:

```

Bin ich im richtigen Verzeichnis?
Nutze ich den richtigen Workspace und Backend-Key?
Bin ich im richtigen AWS-Konto und in der richtigen Region?
Wurde State verschoben oder verloren?
Gab es eine Umbenennung ohne moved-Block oder state mv?
  
```


Terraform-Workflow: Von Code zu kontrollierter Infrastruktur

Planen, prüfen und erst danach ausführen. Das ist der Kern sicherer IaC-Arbeit.



Terraform-Workflow

18.1 Befehle

Befehl	Aufgabe
<code>terraform init</code>	Backend, Provider und Module initialisieren
<code>terraform fmt</code>	HCL einheitlich formatieren
<code>terraform validate</code>	Syntax und interne Konsistenz prüfen
<code>terraform plan</code>	geplante Änderungen anzeigen
<code>terraform apply</code>	Änderungen ausführen
<code>terraform destroy</code>	verwaltete Infrastruktur löschen
<code>terraform output</code>	Outputs aus State anzeigen
<code>terraform graph</code>	Abhängigkeitsgraph im DOT-Format ausgeben

`validate` prüft nicht, ob eine reale Ressource in AWS existiert. Dafür sind `plan` und Provider-API-Zugriff nötig.

18.2 Gespeicherter Plan

```

terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan

```

Ein gespeicherter Plan sorgt dafür, dass genau die geprüften Änderungen ausgeführt werden. In Produktion ist das sicherer als ein ungeprüftes `apply -auto-approve`.

18.3 Wenn Apply teilweise fehlschlägt

Terraform führt nicht automatisch einen vollständigen Rollback durch. Erfolgreich erstellte Ressourcen werden im State festgehalten. Danach:

47. Fehlerursache korrigieren.
48. Erneut `plan` ausführen.
49. Restliche Änderungen kontrolliert anwenden.

Bei einem unerwartet großen Destroy-Plan sofort stoppen und Workspace, Backend, Credentials, Region, Konfiguration und letzte Refactorings prüfen.

19. Terraform State, Remote Backend und Drift

Praxis: Warum der State mehr als eine Bestandsliste ist

Im Code heißt die Ressource `aws_instance.shop_app`. In AWS besitzt sie eine konkrete ID wie `i-0123456789abcdef0`. Der State verbindet beide Welten.

Terraform-Adresse	State-Zuordnung	AWS
<code>aws_instance.shop_app</code>	<code>i-0123456789abcdef0</code>	reale Instanz

Ohne diese Zuordnung könnte Terraform nicht zuverlässig erkennen, welche reale Instanz zu welchem Resource-Block gehört.

Warum ein Remote Backend im Team nötig ist

Liegt der State nur auf Julians Laptop, kennt die CI-Pipeline den letzten Stand nicht. Zwei Teammitglieder könnten gleichzeitig verschiedene States verändern.

Ein zentrales Backend bietet:

- einen gemeinsamen aktuellen State
- Zugriffskontrolle
- Verschlüsselung und Versionierung
- Locking gegen gleichzeitige Schreibvorgänge
- bessere Wiederherstellung

Drift an einem einfachen Beispiel

Terraform beschreibt `versioning = true` für einen S3-Bucket. Jemand schaltet Versioning manuell in der Konsole aus. Der reale Zustand weicht vom Soll ab.

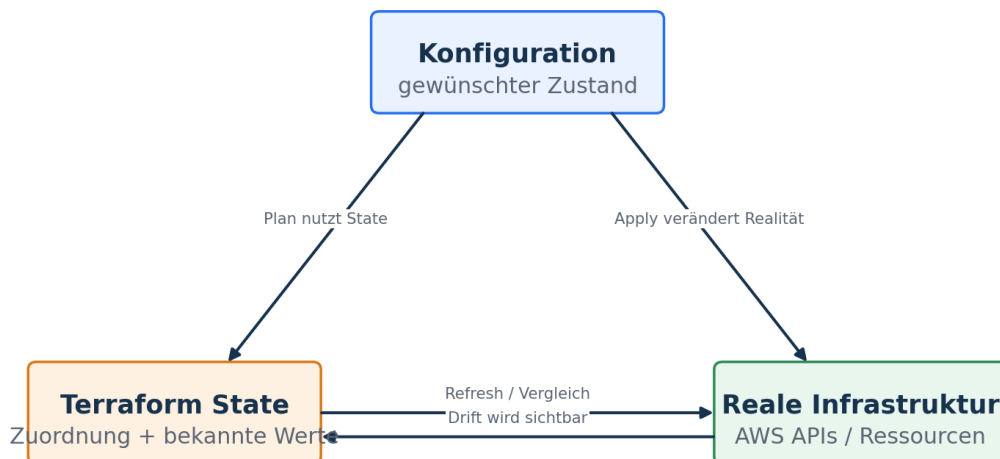
Beim nächsten Plan muss entschieden werden:

50. Die manuelle Änderung war falsch: Terraform stellt `true` wieder her.
51. Die manuelle Änderung soll bleiben: Der Code wird bewusst angepasst.

Drift sollte nicht reflexartig „weggeklickt“ werden. Erst muss geklärt werden, welcher Zustand tatsächlich gewollt ist.

Terraform denkt in drei Zuständen

Probleme entstehen, wenn Code, State und reale Infrastruktur nicht mehr zusammenpassen.



State ist kritisch: schützen, versionieren, verschlüsseln und gleichzeitige Schreibzugriffe sperren.

Terraform-Konfiguration, State und reale Infrastruktur

19.1 Was speichert der State?

Der State verbindet Ressourcenadressen im Code mit realen IDs und bekannten Attributen. Ohne State kann Terraform nicht zuverlässig erkennen, welche Ressource zu welchem Block gehört.

State kann sensible Werte enthalten. Deshalb:

- Zugriff nach Least Privilege
- Verschlüsselung

- Versionierung und Backups
- kein Commit in öffentliche Repositories
- Sperre gegen gleichzeitige Schreibzugriffe

19.2 Local und Remote State

Lokaler State liegt auf dem Rechner des Operators und eignet sich nur für kleine Einzelübungen. Im Team wird ein Remote Backend verwendet. Ein typisches AWS-Muster ist S3 für den State und ein Locking-Mechanismus, im Kurs häufig mit DynamoDB behandelt.

Wichtiger als das konkrete Produkt ist das Prinzip:

- zentraler Zustand
- nur ein schreibender Lauf gleichzeitig
- Versionierung und Wiederherstellung
- verschlüsselter Zugriff
- getrennte States pro System oder Umgebung

19.3 Wichtige State-Befehle

Kleine Einsatzbeispiele

```
terraform state list
```

Zeigt die im State bekannten Terraform-Adressen.

```
terraform state show aws_instance.shop_app
```

Zeigt Attribute einer bestimmten Ressource im State.

```
terraform state mv aws_instance.old aws_instance.shop_app
```

Verschiebt die State-Adresse bei einem Refactoring, ohne die echte Ressource neu zu erstellen.

```
terraform state rm aws_s3_bucket.legacy
```

Entfernt nur die Verwaltung aus dem State. Der reale Bucket bleibt bestehen. Das ist deshalb kein Ersatz für `destroy`. Direktes manuelles Bearbeiten der State-Datei ist riskant. Unterstützte State-Befehle und Backups sind der sicherere Weg.

Befehl	Zweck
<code>terraform state list</code>	Ressourcenadressen auflisten
<code>terraform state show <adresse></code>	Attribute einer Ressource anzeigen
<code>terraform state pull</code>	aktuellen State zur Inspektion abrufen
<code>terraform state mv</code>	Adresse im State verschieben oder umbenennen
<code>terraform state rm</code>	Ressource aus Verwaltung lösen, ohne sie zu löschen
<code>terraform force-unlock</code>	verwaisten Lock lösen; nur wenn sicher kein Lauf aktiv ist

State-Dateien nicht manuell editieren. Unterstützte State-Kommandos, Backups und `moved`-Blöcke sind sicherer.

19.4 Drift

Drift bedeutet, dass reale Infrastruktur von Code und State abweicht, zum Beispiel nach einer manuellen Änderung in der Konsole. `terraform plan` macht diese Abweichung sichtbar.

Mögliche Reaktionen:

- Realität zurück auf den Codezustand ändern
- Code bewusst an die akzeptierte Realität anpassen
- externe Verwaltung einzelner Felder mit `ignore_changes` dokumentieren
- manuell vorhandene Ressource importieren

20. Variablen, Outputs, Locals und sensible Werte

Praxis: Gleicher Code für Development und Production

Der Resource-Block bleibt gleich. Nur die Eingabewerte ändern sich.

```
variable "instance_type" {
  type = string
}
```

```
variable "environment" {
  type = string
  validation {
    condition = contains(["dev", "test", "prod"], var.environment)
    error_message = "environment muss dev, test oder prod sein."
  }
}
```

Development könnte `t3.micro` verwenden, Production eine größere Instanz. Dadurch wird nicht der komplette Code kopiert.

Output als Schnittstelle

Nach dem Apply benötigt die Pipeline vielleicht den DNS-Namen des Load Balancers:

```
output "shop_url" {
  value = aws_lb.shop.dns_name
}
```

Ein Output ist keine Anzeige-Dekoration. Er ist eine definierte Schnittstelle für Menschen, andere Module oder nachgelagerte Automatisierung.

sensitive richtig verstehen

`sensitive = true` verhindert viele Klartextausgaben im Terminal. Der Wert kann trotzdem im State enthalten sein. Der State muss deshalb weiterhin geschützt werden.

20.1 Variablen

```
variable "environment" {
  type = string
  description = "Zielumgebung"

  validation {
    condition = contains(["dev", "test", "prod"], var.environment)
    error_message = "Erlaubt sind dev, test oder prod."
  }
}
```

Wichtige Typen:

- `string`, `number`, `bool`
- `list(T)`, `set(T)`, `map(T)`
- `tuple([...])`, `object({...})`

Ein Set ist ungeordnet und enthält keine Duplikate. Ein Objekt besitzt benannte Felder mit festgelegten Typen.

Variablenwerte können aus Defaults, `terraform.tfvars`, `*.auto.tfvars`, Environment-Variablen oder CLI-Argumenten stammen. In Automatisierung sollten Werte explizit und reproduzierbar gesetzt werden.

20.2 Sensitive

`sensitive = true` verhindert die normale Anzeige in CLI-Logs. Es verschlüsselt den Wert nicht automatisch und entfernt ihn nicht aus dem State. Secrets gehören in einen Secret Store oder geschützte CI/CD-Variablen, nicht in Git.

20.3 Locals und Outputs

Locals geben komplexen Ausdrücken verständliche Namen:

```
locals {
  common_tags = {
    Environment = var.environment
    ManagedBy   = "Terraform"
  }
}
```

Outputs stellen Werte für Menschen, Skripte oder andere Module bereit. Bei Child Modules werden sie über `module.<name>.<output>` gelesen.

21. HCL-Ausdrücke, Funktionen und Iteration

Praxis: Zwei Subnetze ohne doppelte Resource-Blöcke

Mit `for_each` werden benannte Instanzen erzeugt. Die Schlüssel bleiben Teil der Terraform-Adresse.

```
variable "subnets" {
  type = map(string)
  default = {
    app_a = "10.0.11.0/24"
    app_b = "10.0.12.0/24"
  }
}
```

```

}

resource "aws_subnet" "app" {
  for_each = var.subnets
  vpc_id   = aws_vpc.shop.id
  cidr_block = each.value

  tags = {
    Name = each.key
  }
}

```

Die Adressen lauten anschließend beispielsweise:

```

aws_subnet.app["app_a"]
aws_subnet.app["app_b"]

```

Bei `count` wären es numerische Indizes. Wird eine Liste in der Mitte verändert, können Indizes verrutschen. Für benannte, langlebige Ressourcen ist `for_each` deshalb häufig stabiler.

Funktionen haben einen konkreten Zweck

- `merge()` verbindet Tag-Maps.
- `lookup()` liest einen Map-Wert mit Fallback.
- `jsonencode()` erzeugt korrektes JSON für Policies.
- `try()` bietet einen Fallback, wenn ein Ausdruck nicht verfügbar ist.
- `toset()` entfernt Duplikate und erzeugt eine ungeordnete Menge.

Komplexe verschachtelte Ausdrücke gehören oft in verständlich benannte `locals`, damit der Code lesbar bleibt.

21.1 Bedingung und Funktionen

```
instance_type = var.environment == "prod" ? "m6i.large" : "t3.micro"
```

Häufige Funktionen:

Funktion	Zweck
<code>merge()</code>	Maps oder Objects zusammenführen
<code>lookup()</code>	Map-Key mit Default lesen
<code>coalesce()</code>	ersten nutzbaren Wert wählen
<code>try()</code>	Ausdruck mit Fallback bei Fehler
<code>concat()</code>	Listen zusammenführen
<code>join()</code>	Listenelemente zu String verbinden
<code>toset()</code>	Liste in eindeutiges Set umwandeln
<code>jsonencode()</code> / <code>jsondecode()</code>	zwischen HCL-Werten und JSON wechseln
<code>contains()</code>	Element in Collection prüfen
<code>length()</code>	Länge oder Anzahl bestimmen
<code>regex()</code>	Muster prüfen

Komplexe Ausdrücke sollten in Locals zerlegt werden. Lesbarkeit ist wichtiger als eine möglichst kurze Einzeile.

21.2 count und for_each

```

resource "aws_subnet" "private" {
  for_each = var.private_subnets

  cidr_block      = each.value.cidr
  availability_zone = each.value.az
}

```

- `count` eignet sich für eine einfache Anzahl gleichartiger Ressourcen.
- `for_each` eignet sich für stabile, benannte Instanzen aus Map oder Set.

Bei `count` können Positionsänderungen in Listen zu unerwartetem Austausch führen. `for_each` verwendet Keys als Identität. Ändert sich ein Key, sieht Terraform meist eine alte und eine neue Ressource.

`dynamic` erzeugt variable verschachtelte Blöcke. Es sollte nur eingesetzt werden, wenn normale direkte Blöcke nicht ausreichen.

22. Abhängigkeiten und Lifecycle

Praxis: Terraform baut einen Abhängigkeitsgraphen

Eine EC2-Instanz verweist auf eine Subnet-ID:

```
subnet_id = aws_subnet.app["app-a"].id
```

Damit erkennt Terraform automatisch: Das Subnetz muss vorhanden sein, bevor die Instanz erstellt werden kann. Das ist eine implizite Abhängigkeit.

`depends_on` wird benötigt, wenn die Abhängigkeit fachlich existiert, aber nicht durch einen Attributverweis sichtbar wird. Es sollte nicht pauschal überall verwendet werden, weil unnötige Abhängigkeiten Parallelität und Verständlichkeit verschlechtern.

Lifecycle an einem Ersatzvorgang

Ein Target Group-Objekt muss wegen einer unveränderlichen Eigenschaft ersetzt werden.

- Standard: alte Ressource löschen, neue erstellen - mögliche Unterbrechung.
- `create_before_destroy`: zuerst Ersatz erstellen, danach alte Ressource entfernen - wenn Namen, Quoten und Provider dies erlauben.
- `prevent_destroy`: Terraform blockiert einen geplanten Löschvorgang, zum Beispiel bei einer kritischen Datenbank.
- `ignore_changes`: ein bewusst extern verwaltetes Attribut löst keinen Terraform-Update aus.

`ignore_changes` sollte gezielt eingesetzt werden. Sonst wird echter Drift unsichtbar und der Code beschreibt den tatsächlichen Betrieb immer schlechter.

22.1 Implizite Abhängigkeit

Wenn eine Ressource ein Attribut einer anderen referenziert, erkennt Terraform die Reihenfolge automatisch:

```
subnet_id = aws_subnet.private["app-a"].id
```

Das ist besser als unnötiges `depends_on`, weil die fachliche Datenbeziehung sichtbar bleibt.

22.2 Explizite Abhängigkeit

`depends_on` ist sinnvoll, wenn die Abhängigkeit real existiert, aber nicht durch ein Attribut ausgedrückt wird.

Übermäßiger Einsatz macht den Graphen ungenauer und kann unnötig viele Werte erst nach Apply bekannt machen.

22.3 Lifecycle Meta-Arguments

Einstellung	Wirkung	Einsatz
<code>prevent_destroy</code>	blockiert geplantes Löschen	besonders kritische Ressourcen
<code>create_before_destroy</code>	erstellt Ersatz vor dem Löschen	geringere Ausfallzeit bei Austausch
<code>ignore_changes</code>	ignoriert ausgewählte Attributänderungen	extern verwaltete oder bewusst variable Felder
<code>replace_triggered_by</code>	löst Ersatz bei Änderung eines Bezugs aus	gekoppelte Ersatzlogik

`ignore_changes` darf nicht als Methode verwendet werden, um jede Drift zu verstecken. Jede Ausnahme braucht eine klare Begründung.

`-target` führt nur einen Ausschnitt des Graphen aus und sollte auf Ausnahmefälle begrenzt bleiben. Regelmäßiges Targeting kann einen inkonsistenten Gesamtzustand fördern.

23. Terraform-Module

Praxis: Ein VPC-Modul als wiederverwendbare Fabrik

Ohne Modul müsste jedes Team VPC, Subnetze, Route Tables, NAT und Tags erneut schreiben. Ein Modul kapselt dieses Muster.

```

Root Module
|
| Inputs: CIDR, AZs, Name, Environment
v
Child Module "vpc"
|
| erstellt VPC, Subnetze, Route Tables
v
Outputs: vpc_id, public_subnet_ids, private_subnet_ids

```

Ein Aufruf könnte so aussehen:

```

module "shop_vpc" {
  source = "../modules/vpc"

  name = "shop-prod"
  cidr = "10.0.0.0/16"
  azs = ["eu-central-1a", "eu-central-1b"]
}

```

Gute Abstraktion statt maximaler Abstraktion

Ein gutes Modul besitzt eine kleine, verständliche Schnittstelle und sichere Standardwerte. Ein Modul mit fünfzig schwer verständlichen Schaltern ist zwar flexibel, aber oft schwerer zu betreiben als klarer Code.

Versionierte Module verhindern, dass eine ungeprüfte Änderung im zentralen Repository sofort jede Produktionsumgebung verändert.

23.1 Root und Child Module

Das aktuelle Arbeitsverzeichnis ist das Root Module. Ein aufgerufenes Modul ist ein Child Module.

```

module "network" {
  source = "../modules/network"

  vpc_cidr = "10.0.0.0/16"
  subnets = var.subnets
}

output "vpc_id" {
  value = module.network.vpc_id
}

```

Module können aus lokalen Pfaden, Git-Repositories oder einer Registry geladen werden.

23.2 Gute Modulschnittstelle

Ein gutes Modul:

- löst ein klar begrenztes Problem
- besitzt dokumentierte Inputs und Outputs
- verwendet sichere Defaults
- versteckt keine überraschenden Nebenwirkungen
- gibt nur wirklich benötigte Werte aus
- wird versioniert
- ist in Dev/Test vor Produktion geprüft

Nicht jede einzelne Ressource braucht ein eigenes Modul. Ein Modul lohnt sich, wenn ein Muster wiederholt wird oder klare Verantwortung kapselt.

23.3 Versionierung

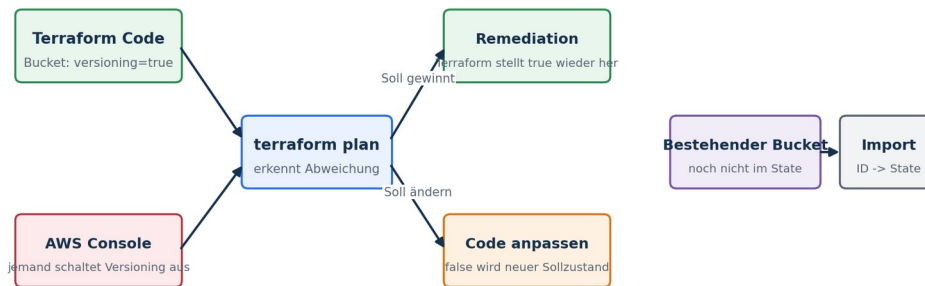
Version Constraints verhindern unkontrollierte Änderungen durch neue Modul- oder Provider-Versionen. Upgrades werden bewusst über `terraform init -upgrade`, Plan und Tests durchgeführt.

Provider Aliases ermöglichen mehrere Konfigurationen desselben Providers, etwa unterschiedliche Regionen oder Konten. Die Root-Konfiguration stellt Credentials und Provider-Kontext bereit und übergibt sie bei Bedarf an Child Modules.

24. Import, Refactoring und Debugging

Terraform Drift und Import

Drift ist eine Abweichung. Import ist das Übernehmen einer bereits vorhandenen Ressource in die Terraform-Zuordnung.



Import schreibt nicht automatisch eine perfekte Konfiguration. Danach muss der Resource-Block so beschrieben werden, dass Code und reale Einstellungen zusammenpassen.

Praxisvisualisierung

Praxis: Ein vorhandener S3-Bucket soll zu Terraform

Der Bucket wurde früher manuell erstellt. Terraform darf ihn nicht neu anlegen, weil der Name bereits existiert und Daten darin liegen.

52. Ein passender `aws_s3_bucket`-Block wird im Code angelegt.
53. `terraform import` verbindet den Resource-Block mit der echten Bucket-ID.
54. Ein Plan zeigt die Unterschiede zwischen Code und Realität.
55. Der Code wird so ergänzt, dass er den gewollten Zustand vollständig beschreibt.

```
terraform import aws_s3_bucket.shop_uploads shop-uploads-prod
terraform plan
```

Import übernimmt die Identität in den State. Er ersetzt nicht die fachliche Modellierung der Ressource.

Fehlerbild: „Terraform will alles neu erstellen“

Häufige Ursachen:

- falsches Backend oder falscher Workspace
- falsches AWS-Konto oder falsche Region
- State nicht gefunden
- Resource-Adresse im Code umbenannt
- Provider-Konfiguration entfernt, obwohl State-Ressourcen sie noch benötigen

Der sichere erste Schritt ist fast nie `apply`. Erst Kontext, State und Plan prüfen.

24.1 Import

Import übernimmt eine bereits existierende Ressource in den State. Import allein erzeugt nicht automatisch eine vollständige gewünschte Konfiguration.

Sicherer Ablauf:

56. passenden Resource Block schreiben
57. reale ID importieren
58. Plan ausführen
59. Code an reale, gewünschte Einstellungen angleichen
60. erst danach Änderungen anwenden

Ein großer Diff nach Import bedeutet meist, dass Code und reale Konfiguration nicht übereinstimmen.

24.2 Refactoring ohne Neuerstellung

Beim Umbenennen oder Verschieben eines Resource Blocks würde Terraform sonst eine alte Ressource löschen und eine neue erstellen. `moved`-Blöcke oder `terraform state mv` erhalten die Zuordnung.

24.3 Debugging

- `TF_LOG=DEBUG` liefert detaillierte Logs. Secrets und sensible Daten können darin auftauchen.

- `terraform console` testet Ausdrücke und Funktionen interaktiv.
- `terraform show -json` liefert Plan oder State für Automatisierung und Policy-Prüfung.
- `terraform providers` zeigt Provider-Abhängigkeiten im Modulbaum.
- `terraform providers lock` unterstützt reproduzierbare Provider-Prüfsummen.

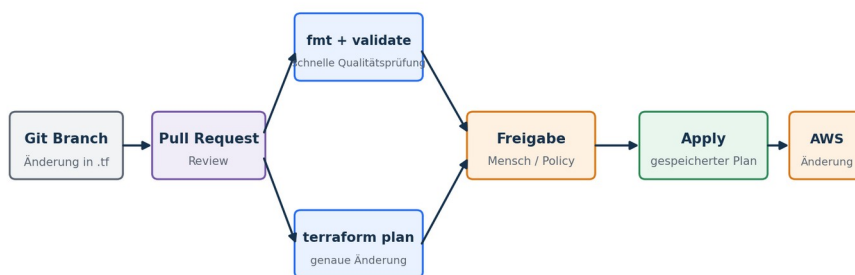
Häufige Fehlerquellen:

- falscher Workspace oder falsches Konto
- Backend nicht korrekt initialisiert
- fehlende Provider-Konfiguration nach Refactoring
- nicht passende Import-Konfiguration
- verwaister State Lock
- externe Änderungen und computed Felder mit permanentem Diff

25. Workspaces und HCP Terraform

Kontrollierter Terraform-Workflow im Team

Code-Review, Plan und Apply werden getrennt. Dadurch ist klar, was geändert wird und wer zugestimmt hat.



Warum besser als ein Laptop-Apply? Die Ausführung ist reproduzierbar, protokolliert, mit festen Rollen abgesichert und nicht von lokalen Dateien oder persönlichen Zugangsdaten abhängig.

Praxisvisualisierung

Praxis: Warum ein kontrollierter Pipeline-Run sicherer ist

Ein lokales Apply hängt von der Umgebung eines einzelnen Rechners ab: lokale Credentials, lokale Provider, aktiver Workspace und möglicherweise nicht eingetragene Dateien.

Ein kontrollierter Teamablauf sieht eher so aus:

```

Git Pull Request
|
+--> terraform fmt + validate
|
+--> Speculative Plan
|
+--> Review / Policy-Prüfung
|
+--> freigegebener Apply
|
+--> Run-Historie und zentraler State
  
```

HCP Terraform beziehungsweise eine vergleichbare Pipeline schafft einen gemeinsamen Ausführungsort, Rollen, Variablen, State und Protokolle.

CLI Workspaces trennen mehrere State-Instanzen derselben Konfiguration. Das kann für einfache, sehr ähnliche Umgebungen nützlich sein. Für stark getrennte Produktionssysteme sind separate Root-Module, Backends oder Konten häufig klarer. Der aktive Workspace ist ein versteckter Kontext, der vor jedem Apply geprüft werden sollte.

25.1 CLI Workspaces

Workspaces halten mehrere States für dieselbe Root-Konfiguration.

```

terraform workspace list
terraform workspace new dev
terraform workspace select prod
terraform workspace show
  
```

Risiko: Der aktive Workspace ist ein versteckter Kontext. Vor Plan oder Apply immer prüfen. Für stark getrennte Produktionssysteme sind getrennte Root-Module, Backends oder Konten oft klarer als viele CLI Workspaces.

25.2 HCP Terraform

HCP Terraform, früher Terraform Cloud, bietet:

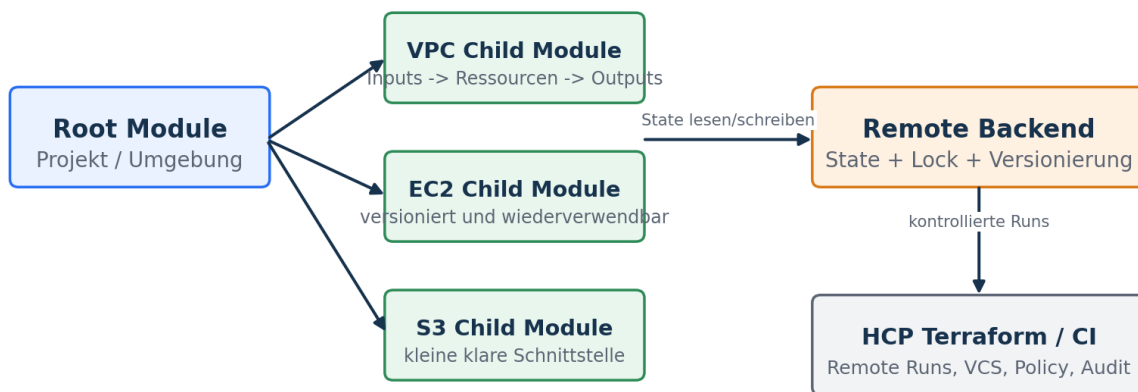
- zentralen State
- Remote Plans und Applies
- VCS-Integration
- Run History und Audit Trail
- Team- und Workspace-Rechte
- geschützte Variablen
- Speculative Plans für Pull Requests
- Policy as Code mit Sentinel

Ein Workspace in HCP Terraform enthält einen eigenen State, Variablen und Run-Kontext. Zugriff sollte nach Rollen und Least Privilege getrennt werden.

25.3 Produktionsworkflow

Terraform-Module und Remote State im Team

Module standardisieren Bausteine. Ein Remote Backend hält den gemeinsamen Zustand.



Module, Remote State und kontrollierte Runs

Die Destroy-Berechtigung kann zusätzlich von normalem Apply getrennt werden. Das reduziert das Risiko eines versehentlichen oder unautorisierten Komplettabbaus.

26. Gesamtarchitektur: Wie die Bausteine zusammenspielen

Vollständiger Request-Pfad: Produktseite öffnen

Der Kunde öffnet `https://shop.example.de/products/4711`.

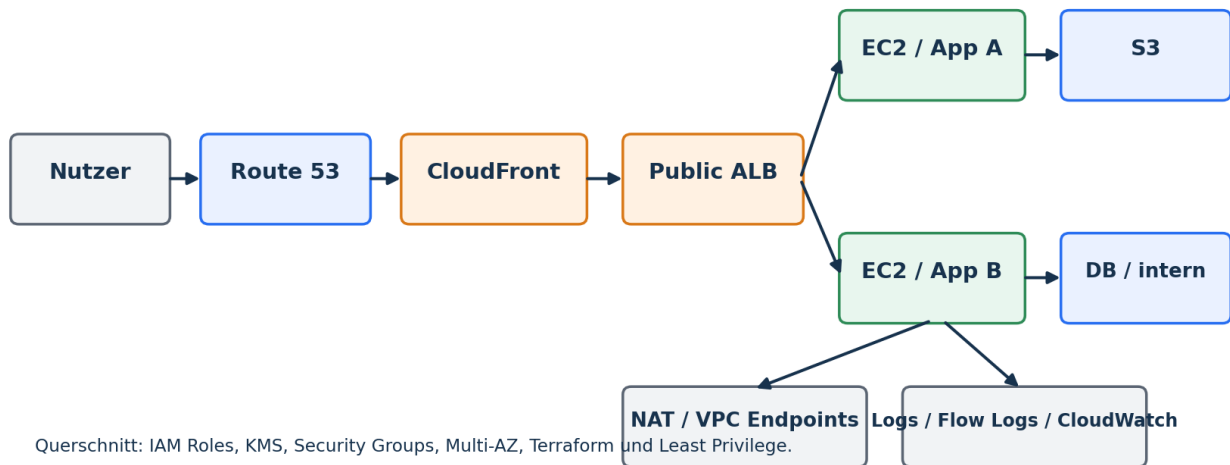
61. **Route 53** löst den Domainnamen auf. Der Record zeigt zum Beispiel auf CloudFront oder den Application Load Balancer.
62. **CloudFront** prüft, ob statische Inhalte bereits im Edge-Cache liegen. Ein Produktbild kann direkt von dort kommen.
63. Eine dynamische Anfrage geht zum **Application Load Balancer** in öffentlichen Subnetzen.
64. Der Load Balancer prüft seine Listener-Regeln und wählt ein gesundes Ziel in einer Target Group.
65. Die Anfrage erreicht einen **EC2-App-Server** im privaten Subnetz. Die App-Security-Group akzeptiert den Port nur vom Load-Balancer-SG.
66. Die Anwendung liest Produktdaten aus der **Datenbank**. Die Datenbank-SG erlaubt den Datenbankport nur von der App-SG.
67. Bilder oder Dokumente werden aus **S3** geladen. Die EC2-Rolle erlaubt nur die erforderlichen Objektaktionen.
68. **CloudWatch** sammelt Metriken und Anwendungslogs. **VPC Flow Logs** liefern bei Netzwerkproblemen Metadaten zum Paketfluss.
69. Änderungen an der Infrastruktur werden über **Terraform** geplant, geprüft und kontrolliert angewendet.

DNS	entscheidet: Wohin verbindet sich der Client?
Routing	entscheidet: Über welchen Netzweg läuft das Paket?
Security Group	entscheidet: Darf diese Verbindung an der Ressource ankommen?
IAM	entscheidet: Darf diese Identität die AWS-API-Aktion ausführen?
Anwendung	entscheidet: Ist die fachliche Anfrage gültig?

Diese Ebenen werden häufig verwechselt. Ein IAM-Problem lässt sich nicht mit einer Route Table lösen. Ein fehlender Netzwerkpfad wird nicht durch AdministratorAccess repariert. Gute Fehlersuche beginnt deshalb mit der Frage: **Auf welcher Ebene scheitert der Vorgang?**

Gesamtbild: Eine typische sichere AWS-Anwendung

Nicht jeder Dienst ist immer nötig. Das Diagramm zeigt, wie die Kursbausteine zusammenspielen.



Gesamtarchitektur

Querschnittliche Kontrollen:

- IAM Roles statt fest gespeicherter Access Keys
- Security Groups zwischen Web-, App- und Datenebene
- NACLs nur für zusätzliche Subnetz-Guardrails
- KMS-Verschlüsselung für sensible Daten
- Multi-AZ für kritische Komponenten
- Flow Logs, CloudTrail und CloudWatch für Nachvollziehbarkeit
- Terraform für wiederholbare Bereitstellung
- S3-Versionierung, Lifecycle und Replikation für Datenhaltbarkeit und Kosten
- Route 53 Health Checks und Failover für passende Multi-Region-Szenarien

26.1 Datenweg einer Webanfrage

Kontrollfragen für jeden Pfeil

Wenn du eine Architektur zeichnest, stelle bei jeder Verbindung fünf Fragen:

70. Wer startet die Verbindung?
71. Welche Quell- und Zieladresse werden verwendet?
72. Welche Route wird gewählt?
73. Welche Netzwerkregeln müssen erlauben?
74. Welche Identität und IAM-Aktion werden danach benötigt?

Beispiel EC2 zu S3:

Start:	EC2-Anwendung
Netzweg:	Gateway Endpoint oder NAT/öffentlicher S3-Endpunkt
Netzschutz:	Route Table, Endpoint Policy, gegebenenfalls SG/NACL des Pfads
Identität:	IAM Role der EC2-Instanz
API-Aktion:	s3:GetObject
Ressource:	definierter Bucket-Pfad

Erst wenn alle benötigten Ebenen stimmen, funktioniert der Gesamtvorgang.

75. Der Benutzer fragt den DNS-Namen bei Route 53 ab.
76. Route 53 liefert CloudFront oder den Load Balancer als Ziel.
77. CloudFront beantwortet Cache-Treffer am Edge. Andere Anfragen gehen zum Origin.
78. Der öffentliche Load Balancer verteilt auf private App-Instanzen in mehreren AZs.
79. Security Groups erlauben nur definierte Verbindungen zwischen den Ebenen.
80. Die App liest Objekte privat über einen VPC Endpoint aus S3 und greift auf die private Datenebene zu.
81. Updates oder externe APIs laufen kontrolliert über NAT oder passende Interface Endpoints.
82. Logs und Metriken liefern Beweise für Fehleranalyse und Audits.

27. DevOps-Fokus: Welche Fähigkeiten zusätzlich wichtig sind

Sinnvolles Praxisprojekt statt einzelner Klickübungen

Ein gutes Lernprojekt verbindet die Themen schrittweise:

83. Eine einfache Webseite auf einer EC2-Instanz starten.
84. Eine eigene VPC mit öffentlichen und privaten Subnetzen aufbauen.
85. Einen Load Balancer vor zwei App-Server setzen.
86. Produktbilder in S3 speichern und per IAM Role lesen.
87. Logs und Metriken in CloudWatch auswerten.
88. Die Infrastruktur zuerst mit AWS CLI nachvollziehen, danach mit Terraform erstellen.
89. State in ein Remote Backend verschieben.
90. Änderungen über Git Branch, Plan und Review ausführen.
91. Einen absichtlichen Fehler einbauen und mit Flow Logs, CloudTrail und App-Logs untersuchen.

Dadurch lernst du nicht nur Dienstnamen. Du lernst, Anforderungen in Architektur, Berechtigungen, Netzwerkpfade und Automatisierung zu übersetzen.

AWS-Dienste allein machen noch keinen DevOps Engineer. Die wichtigsten ergänzenden Fähigkeiten sind:

92. **Linux:** Prozesse, systemd, Dateirechte, Logs, Paketmanagement, Shell.
93. **Netzwerk:** TCP/IP, DNS, Routing, TLS, HTTP, Subnetting und Fehlersuche.
94. **Git:** Branches, Pull Requests, Konflikte, Tags und saubere Commit-Historie.
95. **Scripting:** Bash und mindestens eine Sprache wie Python.
96. **Infrastructure as Code:** Terraform-Module, State, Reviews und Tests.
97. **CI/CD:** Pipelines, Artefakte, Freigaben, Secrets und Rollback-Strategien.
98. **Container:** Docker-Grundlagen, Registry, ECS/Kubernetes-Verständnis.
99. **Observability:** Metriken, Logs, Traces, Alerts und Ursachenanalyse.
100. **Security:** Least Privilege, Rollen, Secret Management, Patch- und Schwachstellenmanagement.
101. **Kosten und Betrieb:** Rightsizing, Tagging, Budgets, Backup und Disaster Recovery.

Die technische Qualität zeigt sich nicht nur daran, dass eine Ressource läuft. Gute Lösungen sind **wiederholbar, sicher, beobachtbar, kosteneffizient und unter Fehlerbedingungen beherrschbar**.

28. Prüfungs- und Praxismerksätze

Von der Formulierung zur technischen Anforderung

Prüfungsfragen enthalten oft Signalwörter. Übersetze sie zuerst in ein Problem, bevor du einen Dienst auswählst.

Formulierung	Eigentliche Anforderung	Häufig passende Richtung
„private Instanz braucht Updates“	ausgehendes IPv4 ohne direkten Inbound	NAT Gateway
„S3 ohne öffentlichen Internetweg“	privater AWS-Service-Zugriff	Gateway Endpoint
„temporäre Rechte ohne Schlüssel“	kurzlebige Credentials	IAM Role + STS
„explizit eine IP blockieren“	Deny am Subnetzrand	NACL oder zentralere Firewall
„wenige VPCs direkt verbinden“	Punkt-zu-Punkt-Routing	VPC Peering
„viele VPCs zentral verbinden“	Hub-and-Spoke	Transit Gateway
„Infrastruktur reproduzierbar“	deklarativer Sollzustand	Terraform / IaC
„vor Apply exakt prüfen“	Änderungsdiff	<code>terraform plan</code>
„bestehende Ressource übernehmen“	State-Zuordnung herstellen	<code>terraform import</code>

Die richtige Antwort ist nicht immer der technisch mächtigste Dienst, sondern der Dienst, der die genannte Anforderung mit dem passenden Betriebsaufwand erfüllt.

AWS

- Explicit Deny gewinnt immer.
- SCP erteilt keine Rechte, sondern begrenzt sie.
- IAM Role liefert temporäre Credentials; keine festen Schlüssel auf EC2 oder Lambda speichern.
- Multi-AZ verbessert Verfügbarkeit; Edge Locations verbessern Auslieferungslatenz.
- Public Subnet bedeutet Route zum IGW, nicht nur eine öffentliche IP.
- Private IPv4-Instanz benötigt NAT für allgemeines ausgehendes Internet, aber keinen NAT für S3 bei Gateway Endpoint.
- Security Group ist stateful und erlaubt nur. NACL ist stateless und kann erlauben oder ablehnen.
- Peering ist nicht transitiv. Transit Gateway ist der Hub für viele Netze.
- EBS ist persistent. Instance Store ist temporär.
- Spot ist günstig, aber unterbrechbar. Capacity Reservation garantiert Kapazität, ist aber kein automatischer Rabatt.
- S3 ist Objektspeicher, EBS Blockspeicher.
- SSE-KMS gibt mehr Kontrolle und Auditierbarkeit als SSE-S3.
- Object Lock Compliance Mode ist WORM und während der Frist nicht übersteuerbar.
- Route 53 Failover ist Active/Passive, Latency Routing wählt die voraussichtlich schnellste Region.

Terraform

- Standardworkflow: `init -> fmt -> validate -> plan -> review -> apply`.
- State ist kritisch und kann sensible Daten enthalten.
- Remote State braucht Zugriffsschutz, Versionierung und Locking.
- `resource` erstellt oder verwaltet, `data` liest nur.
- `for_each` ist für stabile benannte Instanzen oft sicherer als `count`.
- Attributreferenzen erzeugen implizite Abhängigkeiten; `depends_on` nur bei echter unsichtbarer Abhängigkeit.
- `prevent_destroy` schützt, ersetzt aber kein Backup.
- `ignore_changes` gezielt verwenden, nicht zur Verdeckung jeder Drift.
- Import verbindet reale Ressource mit State; passende Konfiguration bleibt nötig.
- Bei einem unerwartet großen Plan zuerst Kontext prüfen, nicht anwenden.
- Module brauchen kleine, klare Schnittstellen und feste Versionen.
- Workspaces sind State-Kontexte; der falsche Workspace kann echte Infrastruktur treffen.

29. Kompakte Befehlsübersicht

Befehle nicht nur auswendig lernen

Ein Befehl ist leichter zu behalten, wenn klar ist, welche Ebene er anspricht:

```
aws ...    -> ruft eine AWS-API auf
ssh ...    -> baut eine Netzwerk- und Host-Sitzung auf
git ...    -> verwaltet Quellcode und Versionshistorie
terraform ... -> vergleicht und verändert deklarierte Infrastruktur
```

Vor jedem problematischen Befehl lohnt sich eine Kontextprüfung:

```
pwd                # Bin ich im richtigen Projekt?
git status          # Welche Codeänderungen liegen vor?
aws sts get-caller-identity # Welches AWS-Konto und welche Identität nutze ich?
terraform workspace show # Welcher State-Kontext ist aktiv?
terraform plan       # Was würde sich wirklich ändern?
```

AWS CLI

```
aws --version
aws configure
aws sts get-caller-identity
aws ec2 describe-instances
aws s3 ls
```

Git

```
git status
git pull
git add .
git commit -m "Beschreibung"
git push
```

Terraform

```
terraform version
terraform init
terraform fmt -recursive
terraform validate
terraform plan -out=tfplan
terraform show tfplan
terraform apply tfplan
terraform output
terraform state list
terraform state show <adresse>
terraform workspace show
terraform destroy
```

30. Glossar

Begriffe als Zusammenhänge lernen

- **API:** Technische Schnittstelle, über die Console, CLI, SDK und Terraform AWS-Aktionen auslösen.
- **ARN:** Eindeutiger Name einer AWS-Ressource, häufig in IAM-Policies verwendet.
- **Principal:** Identität, die eine Anfrage stellt.
- **ENI:** Virtuelle Netzwerkkarte mit IP-Adressen und Security Groups.
- **Egress:** Ausgehender Verkehr.
- **Ingress:** Eingehender Verkehr.
- **Stateful:** Rückverkehr einer erlaubten Verbindung wird erkannt und automatisch berücksichtigt.
- **Stateless:** Hin- und Rückrichtung werden unabhängig geprüft.
- **Backend:** Speicher- und Ausführungsumgebung für Terraform State beziehungsweise Runs.
- **Provider:** Terraform-Plugin, das Ressourcen über die API eines Zielsystems verwaltet.
- **State:** Zuordnung zwischen Terraform-Adressen und realen Ressourcen.
- **Drift:** Realität, Code und/oder State stimmen nicht mehr überein.
- **Idempotent:** Wiederholte Ausführung führt bei unverändertem Sollzustand nicht zu unnötigen Änderungen.
- **Blast Radius:** Bereich, der von einem Fehler oder kompromittierten Zugriff betroffen sein kann.

Begriff	Kurzbedeutung
AMI	Startvorlage für EC2 mit OS und Baseline
ARN	eindeutiger Ressourcenname in AWS
AZ	isolierte Rechenzentrumszone innerhalb einer Region
CIDR	Schreibweise für IP-Netzgröße
Drift	Abweichung zwischen gewünschter und realer Infrastruktur
EBS	persistenter Blockspeicher für EC2
Edge Location	Standort zur schnellen Inhaltsauslieferung
ENI	virtuelle Netzwerkkarte
IAM	Identitäts- und Berechtigungsverwaltung
IGW	Internet Gateway für eine VPC
KMS	Schlüsselverwaltung und Verschlüsselungsdienst
NACL	stateless Subnetz-Firewall
NAT	Übersetzung für ausgehenden privaten IPv4-Verkehr
OU	Container für Konten in AWS Organizations
SCP	organisationsweite Berechtigungsobergrenze
Security Group	stateful Firewall an ENI/Ressource
State	Terraform-Zuordnung zwischen Code und realen Ressourcen
STS	Dienst für temporäre AWS-Credentials
VPC	logisch isoliertes Netzwerk in AWS
VPC Endpoint	privater Zugang von einer VPC zu AWS-/PrivateLink-Diensten

Schluss

Lernhinweis: Zeichne die Datenwege beim Wiederholen selbst nach. Wenn du zu jedem Pfeil sagen kannst, wer die Verbindung startet, welche Route, Security-Regel und IAM-Berechtigung beteiligt ist, hast du den Zusammenhang verstanden.

Die zentrale Idee des gesamten Kurses lautet: **AWS stellt Bausteine bereit, aber eine gute Architektur entsteht durch saubere Grenzen und kontrollierte Abläufe.** Netzwerk, Identität, Speicher, Compute und Terraform sind keine getrennten Inseln. Sie greifen ineinander.

Beim Nachschlagen solltest du deshalb immer vier Fragen stellen:

102. Wo liegt die Ressource - global, Region, AZ, VPC oder Subnetz?
103. Wer darf sie benutzen und über welchen Netzwerkpfad?
104. Welche Teile verwaltet AWS und welche bleiben bei mir?
105. Wie kann ich die Konfiguration wiederholbar, beobachtbar und sicher ändern?